

Mobile Forensics Process Flows

Android / iOS acquisition → decryption → evidence → reports

Complete acquisition, artifact, recovery and report diagrams, including encrypted chats and files.

Includes original .crypt and CRYPT5/7/8/9/10/11/12/14/15, private key intake, derived file registration and executable synthetic test cases.

Matching keys and real acquired bytes are required. Live phone/GPU acceptance remains pending.

Contents and diagram key

Mobile / Acquisition-to-report overview	3
The 13 controlled stages	4
01-02 / Intake and acquisition route	5
03 / Live Android extraction	6
03 / iOS, UFED and backup import	7
04 / Files, DB companions and artifact identities	8
05 / Encryption routing and capability gates	9
WhatsApp / Private key capture into Intake	10
05 / WhatsApp authenticated backups	11
05 / WhatsApp conversations and attachments	12
05 / Messages, email, browsing and documents	13
06 / Deleted chats, photos and retained recovery	14
07 / OCR and searchable text	15
08 / Image and video evidence review	16
09 / Mobile inventory and Suspicious Activity	17
10 / Text-only RAG chunking	18
11-12 / Entities and evidence graph	19
13 / Mobile validation and coverage	20
The single progressAgent supervisor	21
Report / Intake, scope and evidence selection	22
Report / Grounded section generation	23
Report / Media evidence and observations	24
Report / Common section and page structure	25
Report / Shared layout, review and export	26
Other encrypted files / External decryption and import	27
Encryption support and acquisition matrix	28
Live phone / GPU acceptance and performance	29
WhatsApp / Historical CRYPT formats	30
WhatsApp / Derived payloads before artifact parsing	31
WhatsApp / Supported formats and executable test case	32

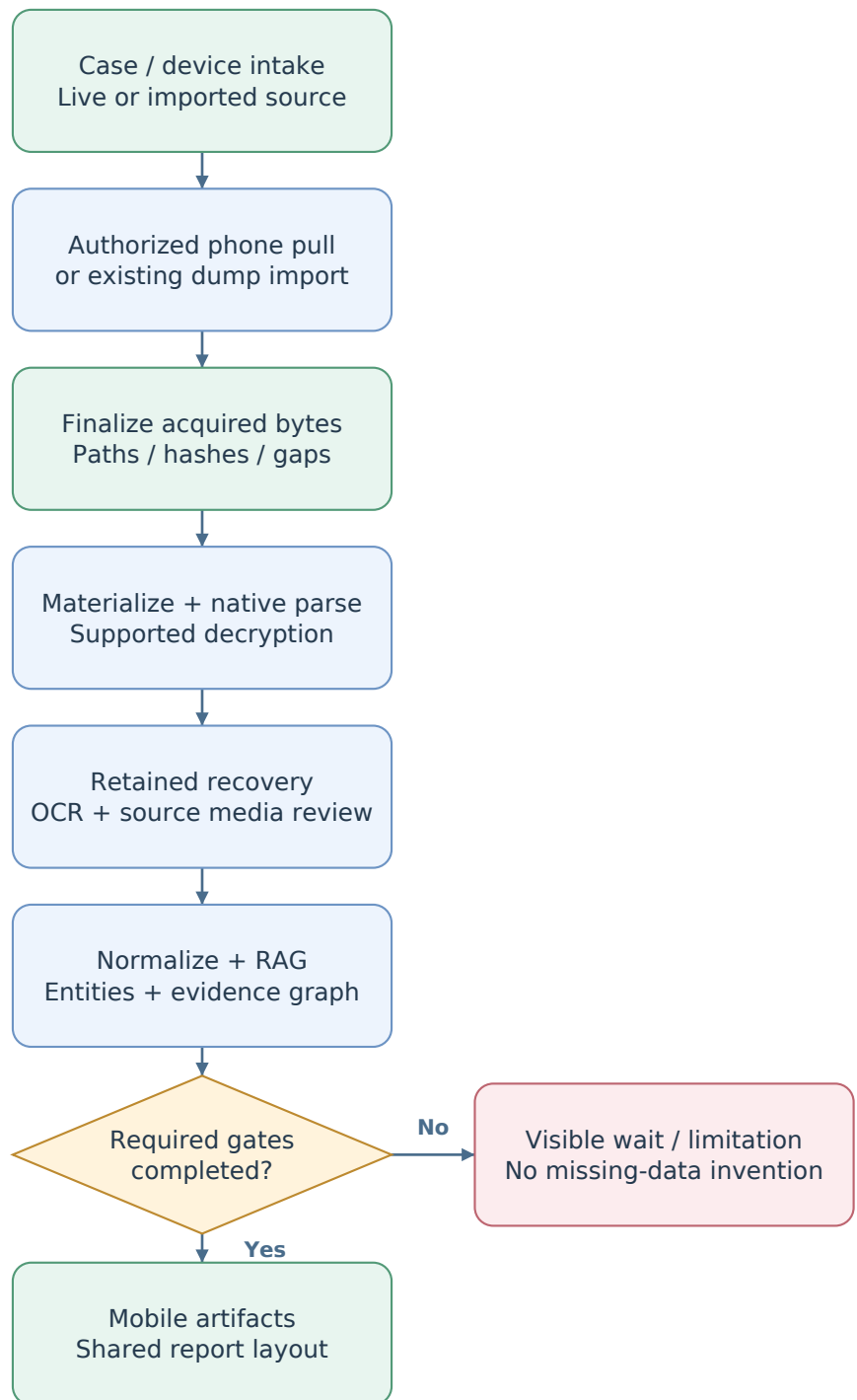
Diagram key

 Process	 Gate / branch	 Retained evidence
 Key / crypto	 Gap / failure	 External workflow

Solid arrows show implemented transitions. Dashed arrows show an explicitly external or conditional path. A gap is evidence of a limitation; it does not establish that the requested data never existed.

Mobile / Acquisition-to-report overview

Android and iOS use Mobile service boundaries and their own stores/queues. The shared forensic stages begin after acquisition handoff.



Priorities WhatsApp chats/media, browsing, pictures/videos/audio, downloads/documents and relevant app records are prioritized, then the remaining accessible content is inventoried.

Coverage A normal unprivileged Android logical pull does not guarantee private databases, encryption keys or deleted bytes. Record actual access and acquired-file counts per family.

Implementation: backend/app/services/forensic_serial_mobile.py; mobile_acquire/; mobile_adapters/

The 13 controlled stages

The same serial controller governs Mobile evidence after acquisition handoff. One stage owns the job at a time; bounded workers may operate inside a stage.

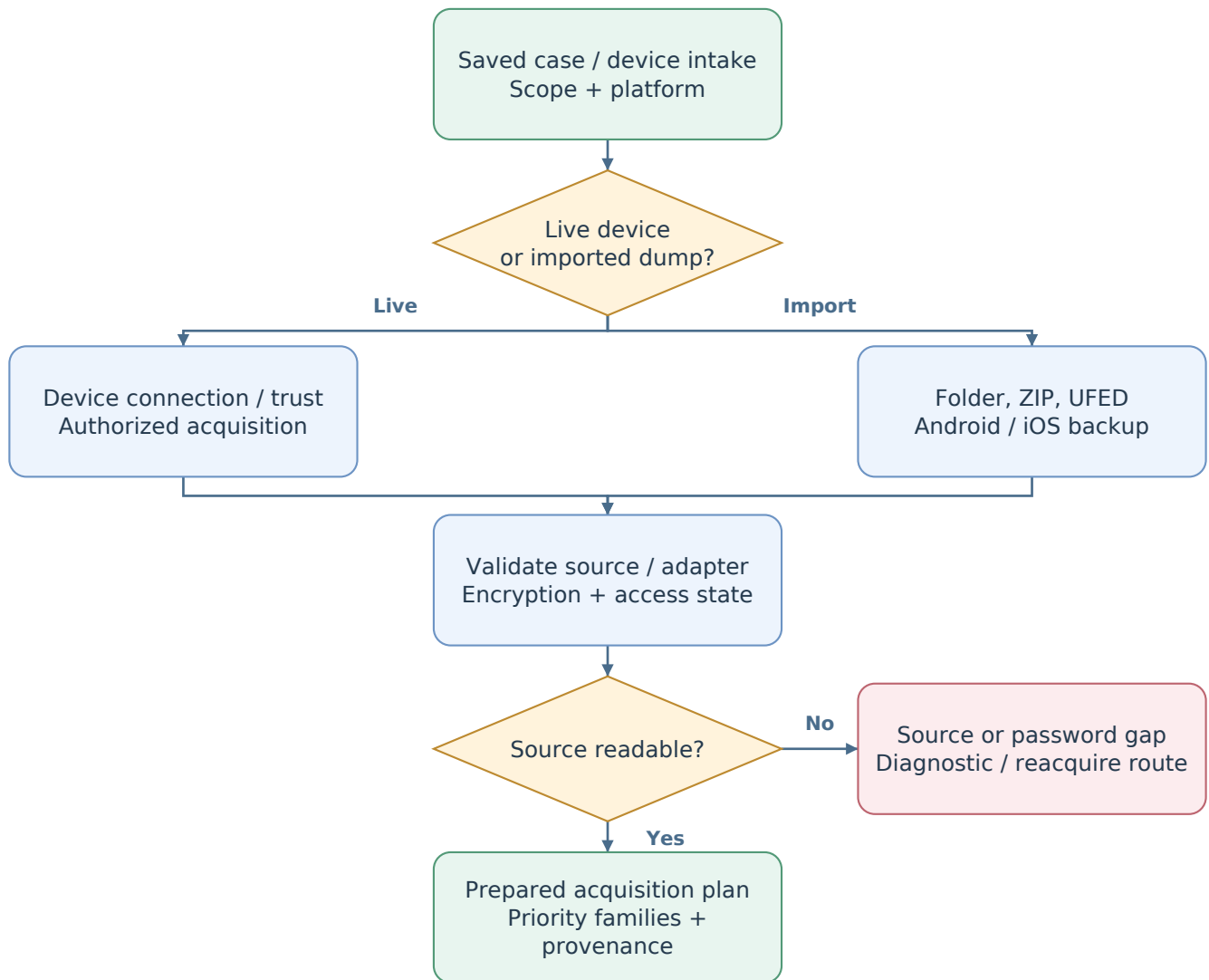
Stage / key	Process agent	Required output
01 intake	segments_agent	Source validation and an extraction plan
02 source_prepare	virtual_disk_agent	Read-only source access / prepared mobile acquisition
03 extraction	extraction_agent	Acquisition completion and a finalized manifest
04 materialize	materialize_agent	Manifest files become durable artifact records
05 parse	parse_agent	Structured records, text and decrypted WhatsApp databases
06 recovery	recovery_agent	Retained deleted bytes and residual candidates
07 ocr	ocr_agent	Text from acquired image/document pixels
08 media_review	media_review_agent	Source images and sampled video frames with descriptions
09 inventory	artifacts_agent	Normalize, classify and correlate evidence
10 chunk	chunk_agent	Text-only RAG: 2500 characters; 800 overlap
11 enrichment	entity_agent	Source-backed entities and enrichment
12 graph	neo4j_agent	Idempotent evidence relationships
13 validation	validation_agent	Stage gates, counts and explicit coverage gaps

State meanings Ready means the configured gates completed. `coverage_complete` and exception counters still describe missing or inaccessible evidence. A completed acquisition does not prove that every possible artifact was captured.

Implementation: `backend/app/services/forensic_serial_pipeline.py`; `forensic_serial_stages.py`; `forensic_serial_policy.py`

01-02 / Intake and acquisition route

Select a route that can supply the required source bytes. A parser can analyze acquired data; it cannot replace absent acquisition access.



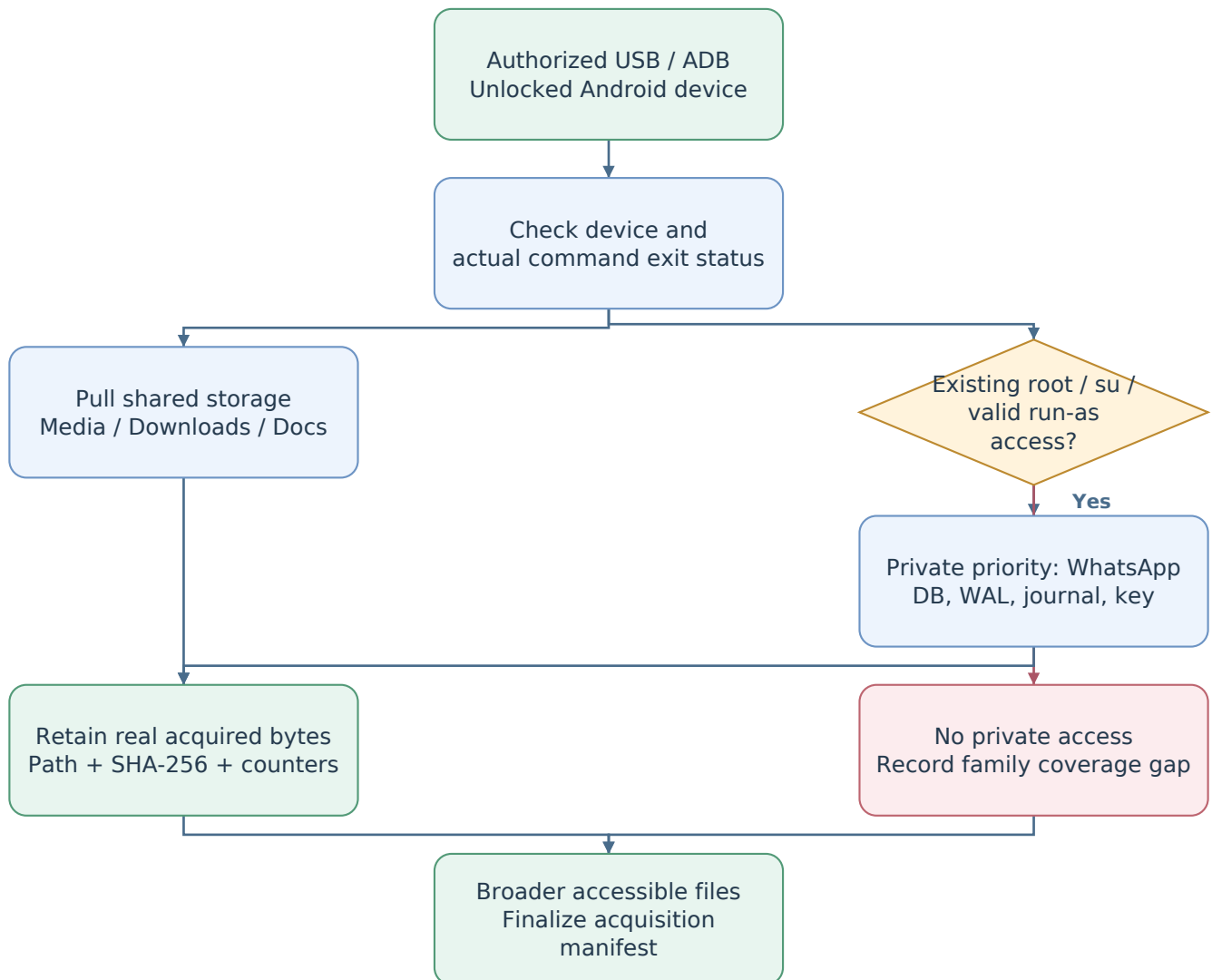
Platform isolation Android and iOS jobs stay inside Mobile services. A Mobile job must not be redirected through the Disk runtime merely to bypass an acquisition or parsing error.

Private-data requirement For inaccessible private apps, use an authorized acquisition that actually contains those databases and matching keys. The application does not root, exploit or unlock a phone.

Implementation: backend/app/services/mobile_adapters/registry.py; mobile_acquire/adapters/; forensic_serial_mobile.py

03 / Live Android extraction

Read accessible WhatsApp keys before large media copies. Logical/Backup routes use a key-only step; File System routes prioritize keys before walking private app files.



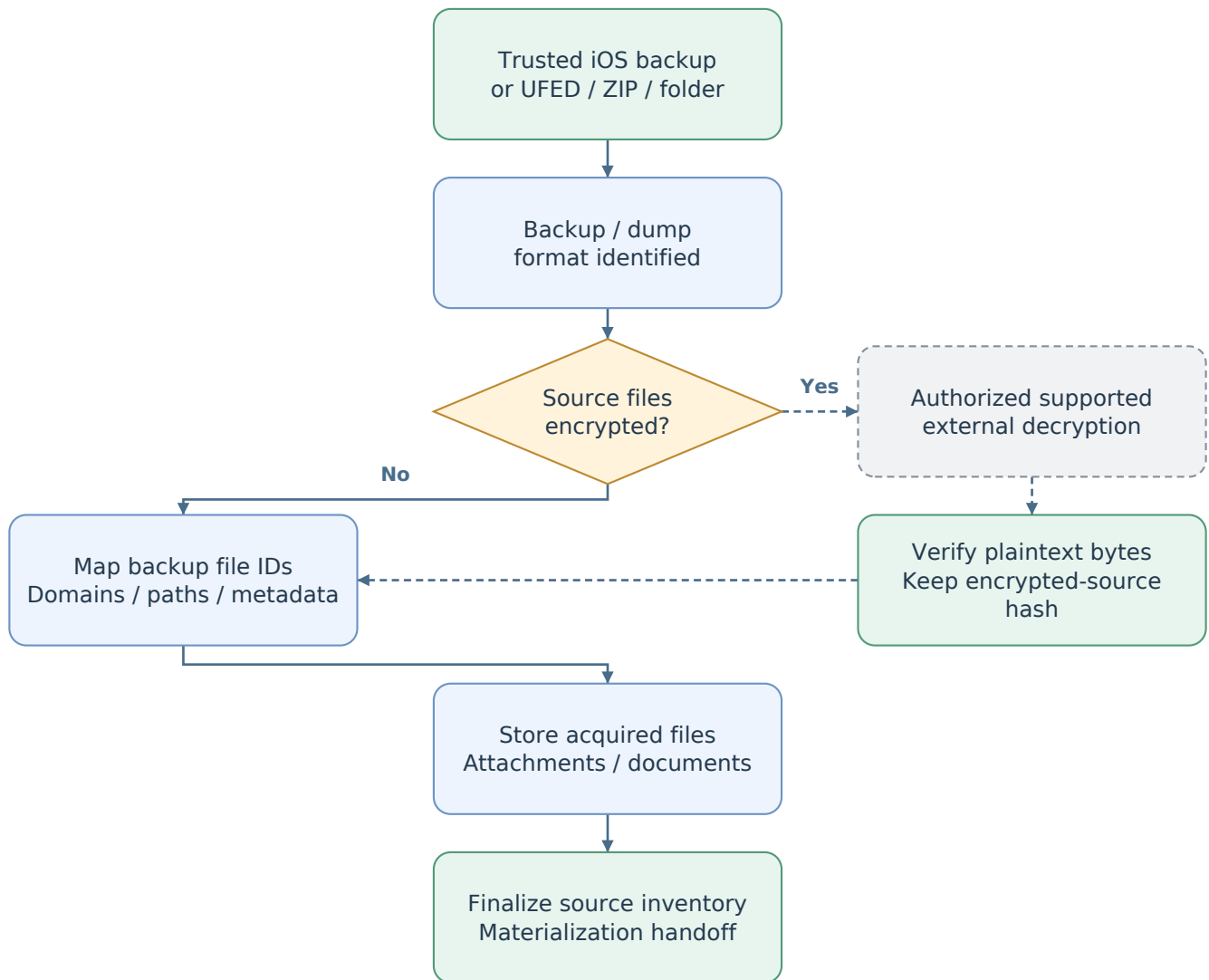
Private priority Check standard and multi-user com.whatsapp paths plus WhatsApp Business (com.whatsapp.w4b). Collect databases and companions together when available.

Invalid key files Reject acquisition-error text such as “run-as: package not debuggable” using command status and binary validation. Saving that error as a key does not make decryption possible.

Implementation: backend/app/services/mobile_acquire/adapters/android_adb.py;
mobile_forensic/whatsapp_crypt.py

03 / iOS, UFED and backup import

Imported packages are mapped into a source inventory before native parsing. Retain each package's original paths and metadata.



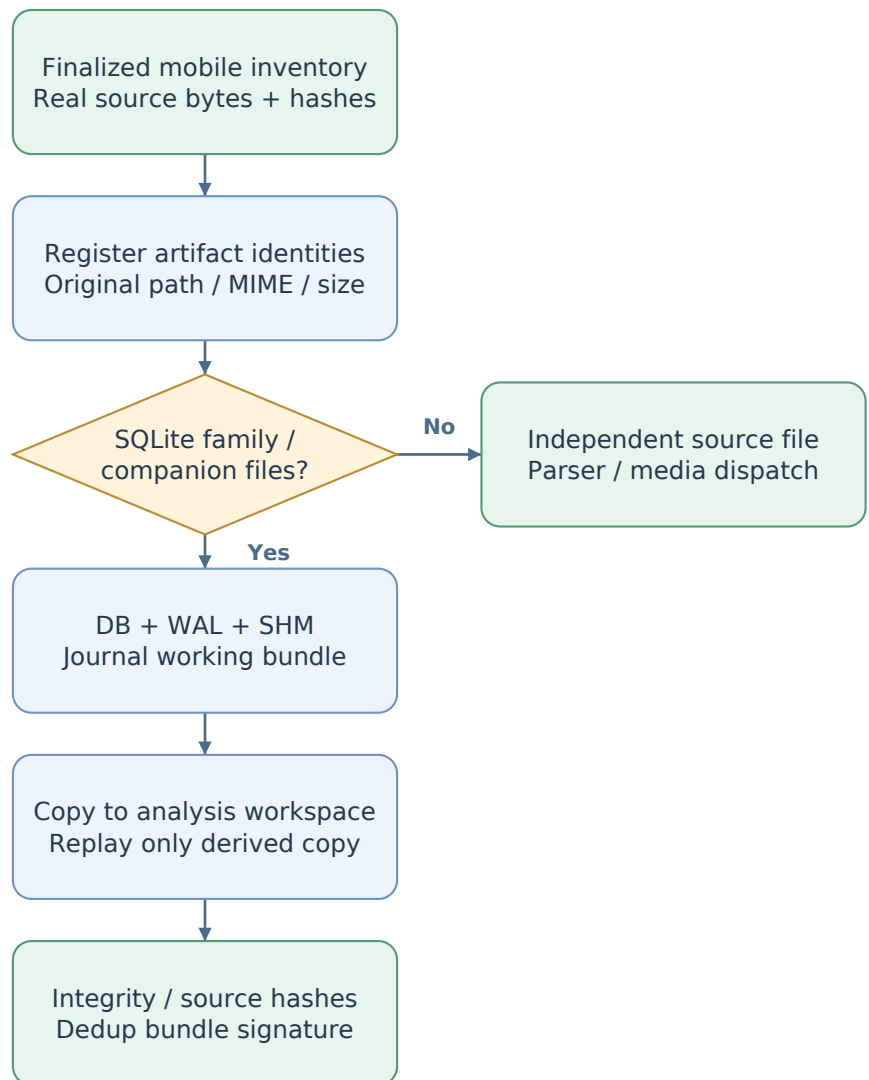
Encrypted iOS import The current importer does not implement complete encrypted-backup decryption merely because a password is supplied. Use supported external decryption, verify the result, then import plaintext.

Import precision UFED logical/full-filesystem exports, Android backup and folder imports provide only their included files. A manifest entry with no acquired bytes stays a reference or a coverage gap.

Implementation: backend/app/services/mobile_adapters/import_ios_backup.py; import_ufed.py; import_zip.py; import_android_backup.py

04 / Files, DB companions and artifact identities

Materialization joins related SQLite source files into a working bundle while preserving originals.



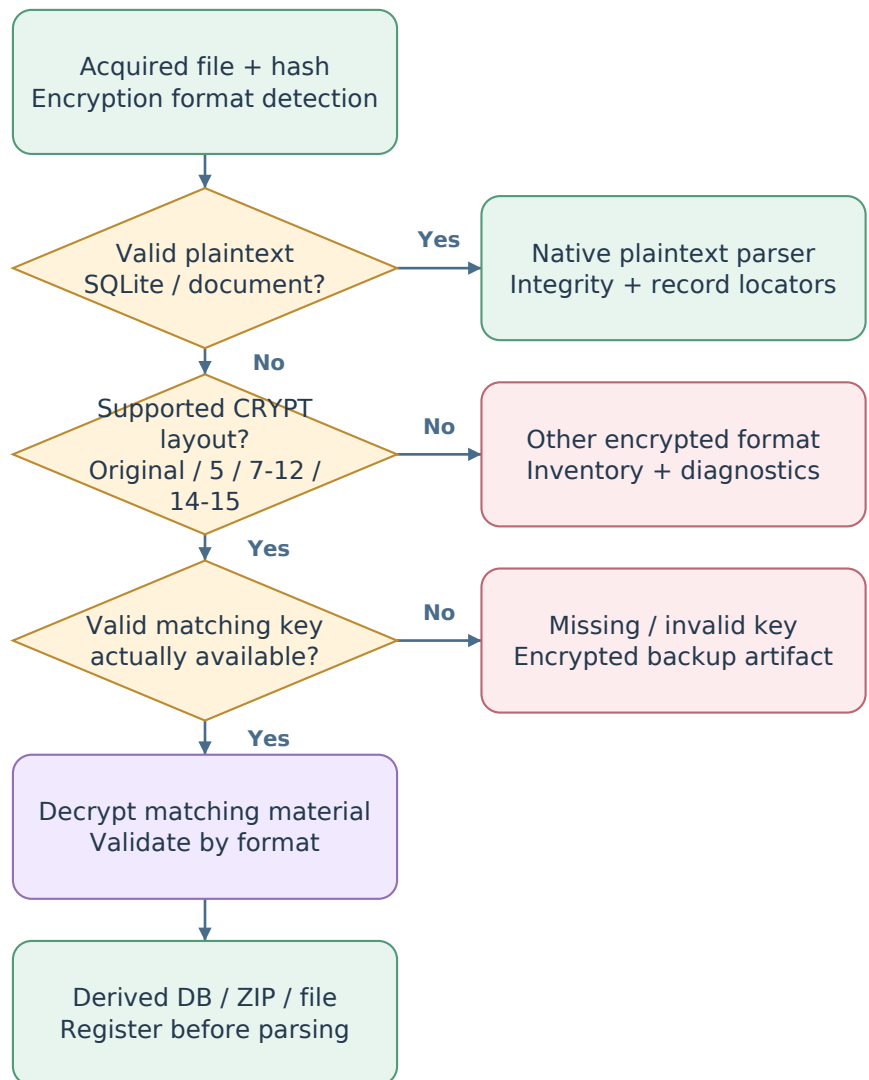
Companions matter A DB-only pull can omit recent messages that remain in a WAL or journal. Acquire, associate and replay companions in a derived workspace rather than modifying source evidence.

Scale Bounded DB bundles and batched persistence support full normalized inventories. Large row sets are streamed and UI-paged rather than truncated to one preview page.

Implementation: backend/app/services/forensic_serial_mobile.py; mobile_forensic/parsers/_sqlite_util.py; mobile_artifact_inventory.py

05 / Encryption routing and capability gates

Identify the actual encrypted container. WhatsApp backup keys are not Android filesystem keys or universal app decryption keys.



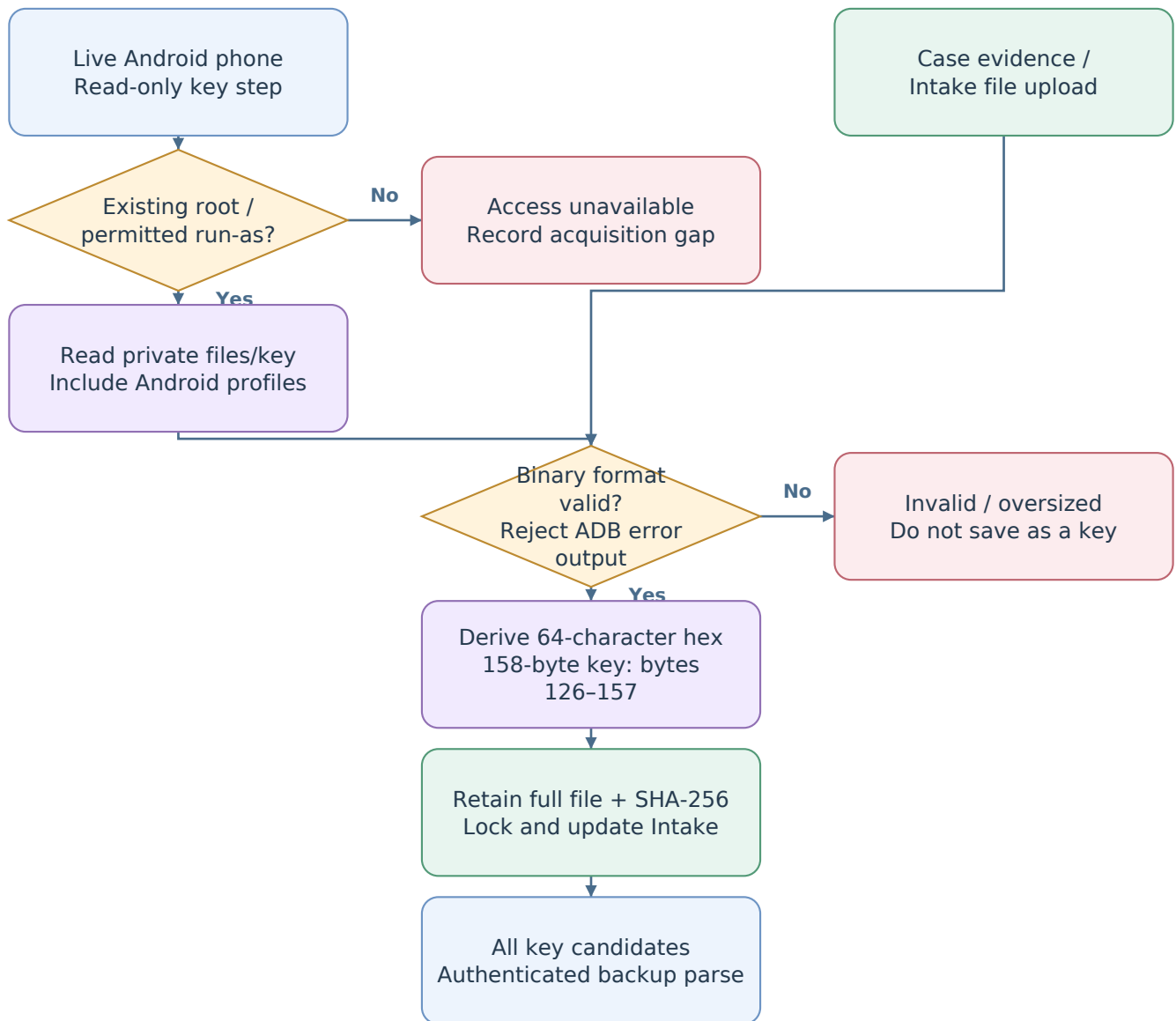
Other formats Signal SQLCipher, encrypted iOS backups, device FBE, password-protected documents/archives and encrypted mail require a supported authorized external route with the correct key or password in this package.

No bypass If source bytes or matching secrets are absent, retain the encrypted artifact and a precise gap. An APK/DEX/ART cache is program code, not recovered chat content.

Implementation: backend/app/services/mobile_forensic/whatsapp_crypt.py; mobile_forensic/parsers/messaging.py; mobile_adapters/import_ios_backup.py

WhatsApp / Private key capture into Intake

Collect the key file into the case acquisition, or upload an already-acquired file in Intake. Keep the full file and record its provenance.



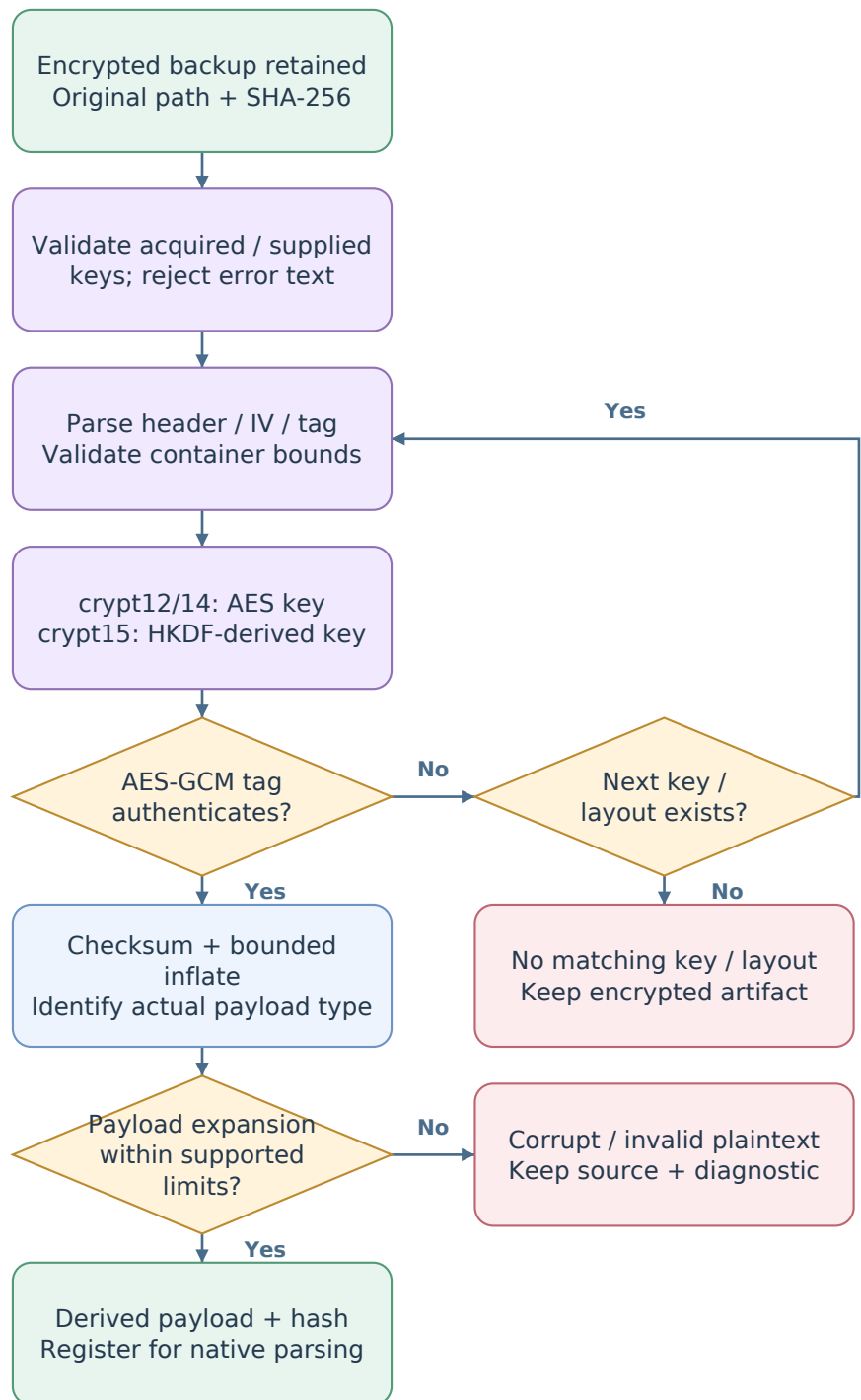
Intake behavior Auto-capture follows materialization. Capture from case evidence and Upload acquired key file are available in Intake. Existing examiner keys are preserved; captured sources remain available as candidates.

Verification boundary A 64-character hex value is saved; raw secrets stay out of API metadata, logs and reports. Format validation alone does not prove a matching backup. After extraction, new key files use an explicit Intake upload. crypt15 uses its separate backup-key derivation.

Implementation: mobile_acquire/privileged_app_pull.py; mobile_forensic/key_intake.py; routers/report.py

05 / WhatsApp authenticated backups

Modern backups authenticate before payload expansion. Payloads can contain SQLite databases, incremental ZIPs, settings, images or binary data.



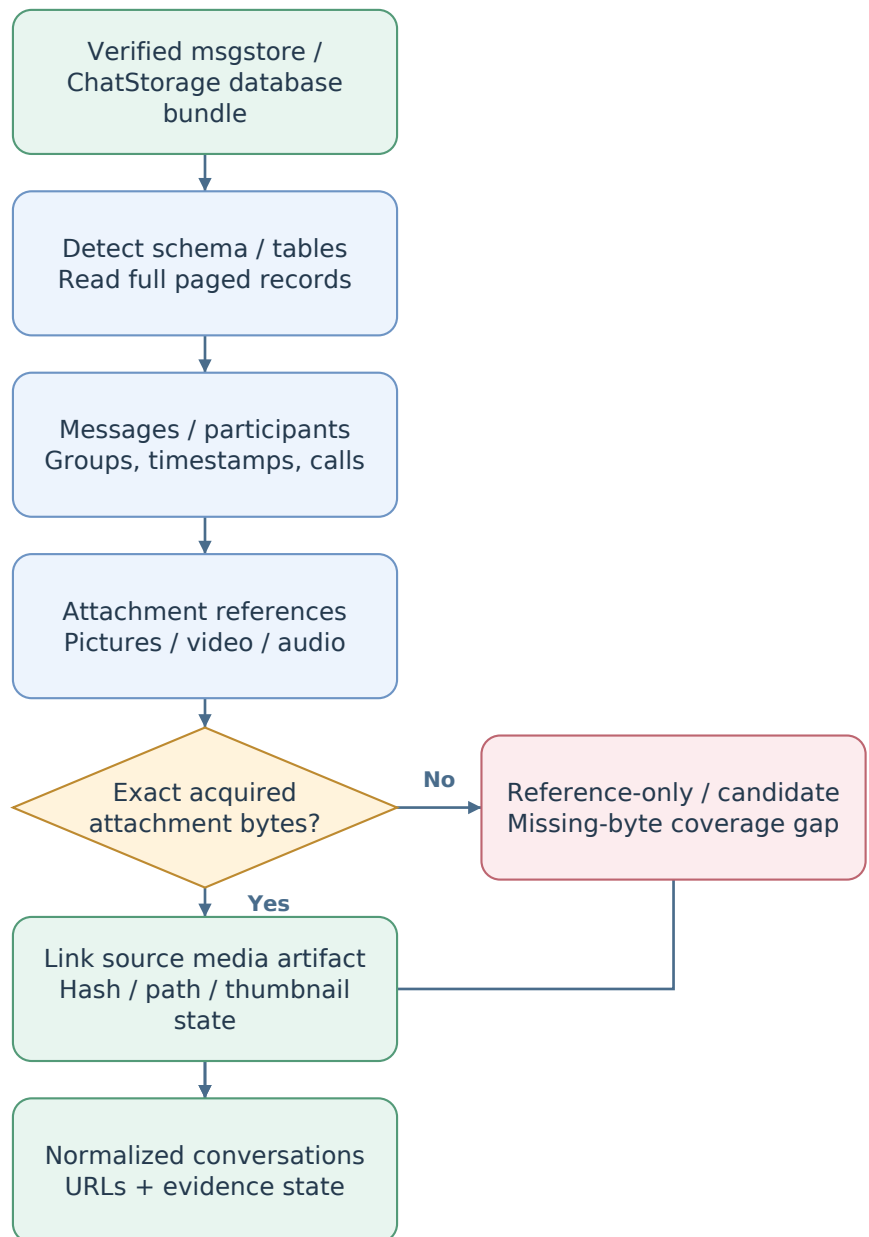
Key formats A 158-byte Android key file supplies its last 32 bytes (offset 126). Raw 32-byte / 64-hex keys and recognized Java byte-array envelopes are validated.

Crypto boundary CRYPT15 uses HKDF-SHA256. Every key candidate is tried per source, including Business and Android profiles. Protobuf IVs avoid repeated historical guesses; source/key-set cache avoids repeat decryption.

Implementation: backend/app/services/mobile_forensic/whatsapp_crypt.py; whatsapp_derivation.py

05 / WhatsApp conversations and attachments

Plaintext or successfully decrypted chat databases enter the native conversation parser with companion-file context.



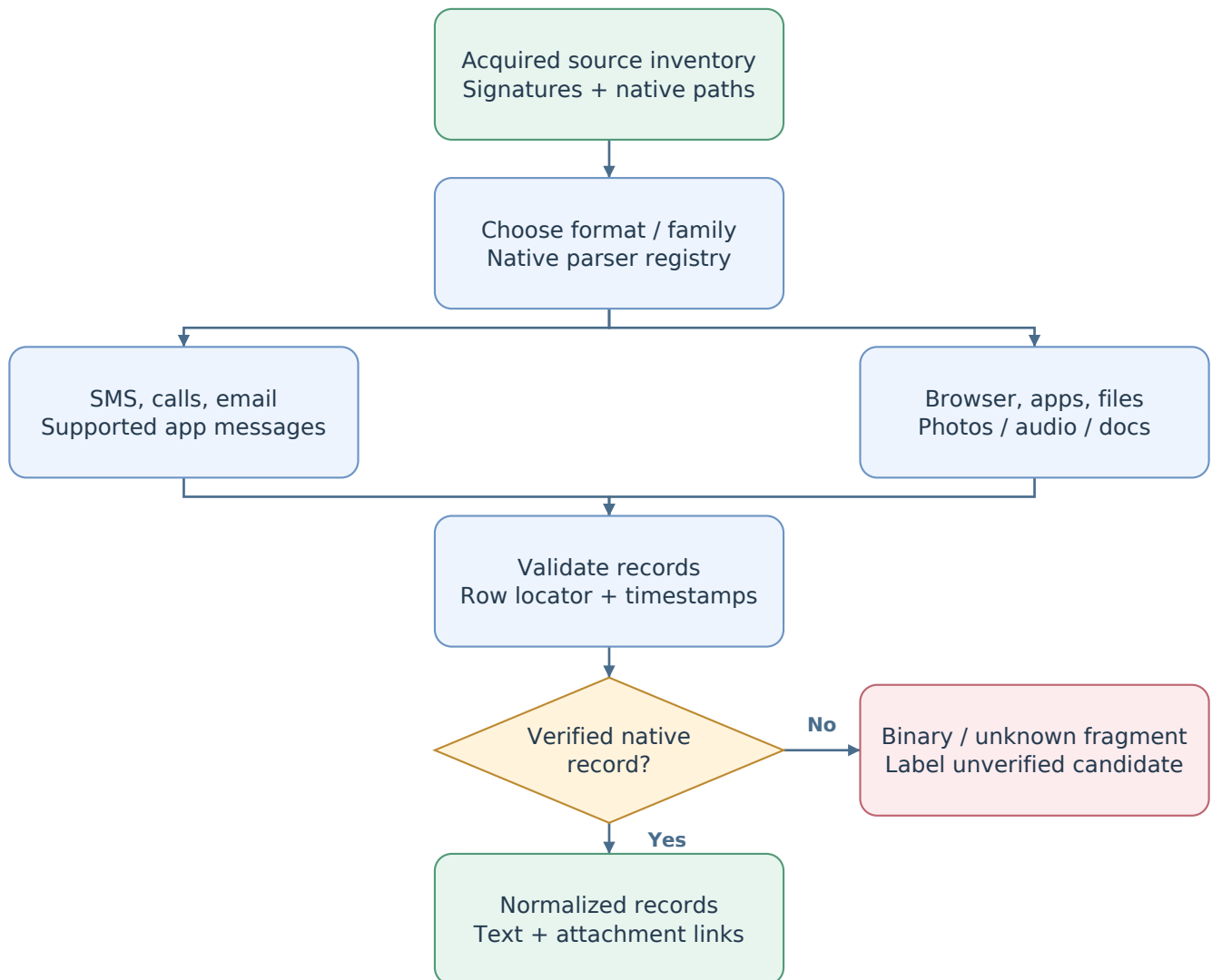
States Keep current database rows, backup-historical rows and explicit deleted markers distinct. An older backup message is not automatically a deleted message.

Completeness Parse all available records rather than a UI preview count. Bundle hashing includes companions; retries must not hide new WAL content or duplicate unchanged snapshots.

Implementation: backend/app/services/mobile_forensic/parsers/whatsapp_modern.py; whatsapp_export.py; whatsapp_evidence_browse.py

05 / Messages, email, browsing and documents

The parser registry records multiple evidence families. Each family is limited to its actual acquired source and supported schema.



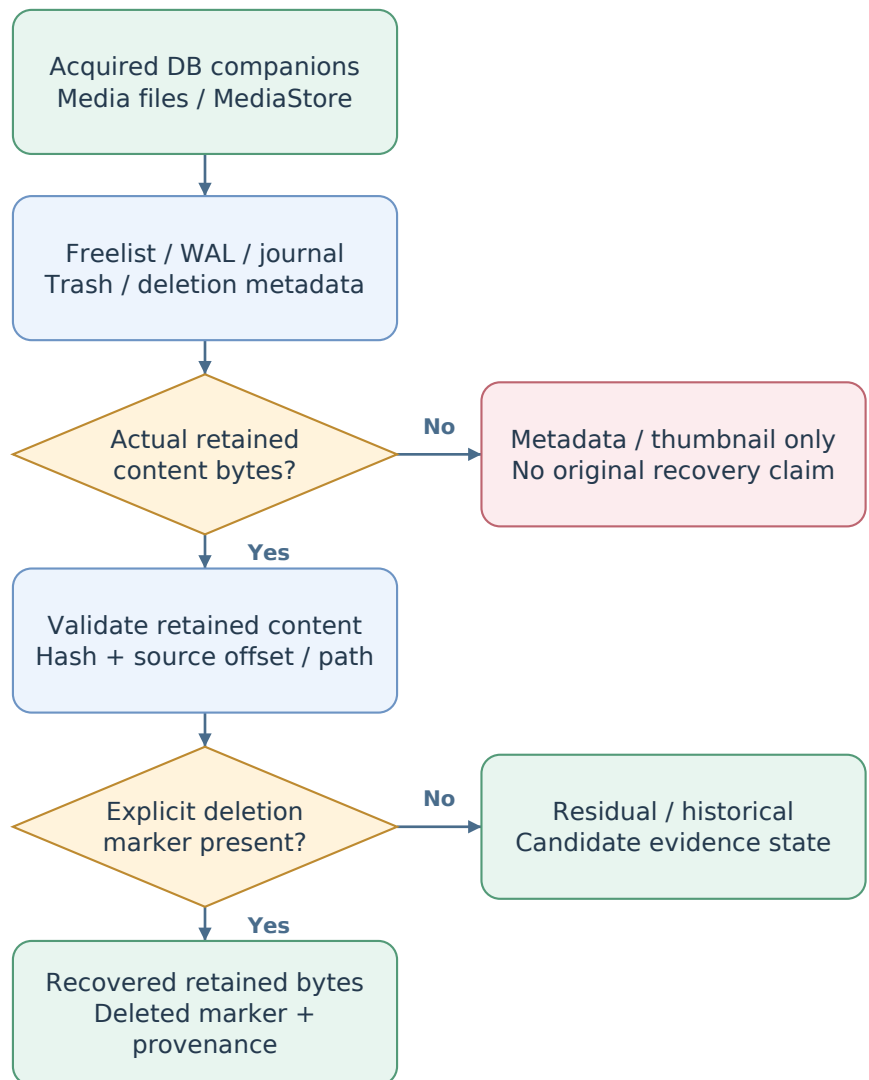
Format families EML/EMLX/MSG/MBOX and attachments; office/PDF/text; CSV/TSV, JSON/JSONL/NDJSON, XML and plist/bplist; supported native SQLite and media metadata, including extensionless files.

Interpretation Browsing visits, bookmarks, downloads and copied URLs retain distinct event kinds. App inventory identifies available packages/accounts; the presence of an app alone does not prove suspicious activity.

Implementation: backend/app/services/mobile_forensic/parsers/; forensic_priority_evidence.py; mobile_forensic/package_inventory.py

06 / Deleted chats, photos and retained recovery

Recovery distinguishes actual retained bytes, explicit deletion records and unverified residual candidates.



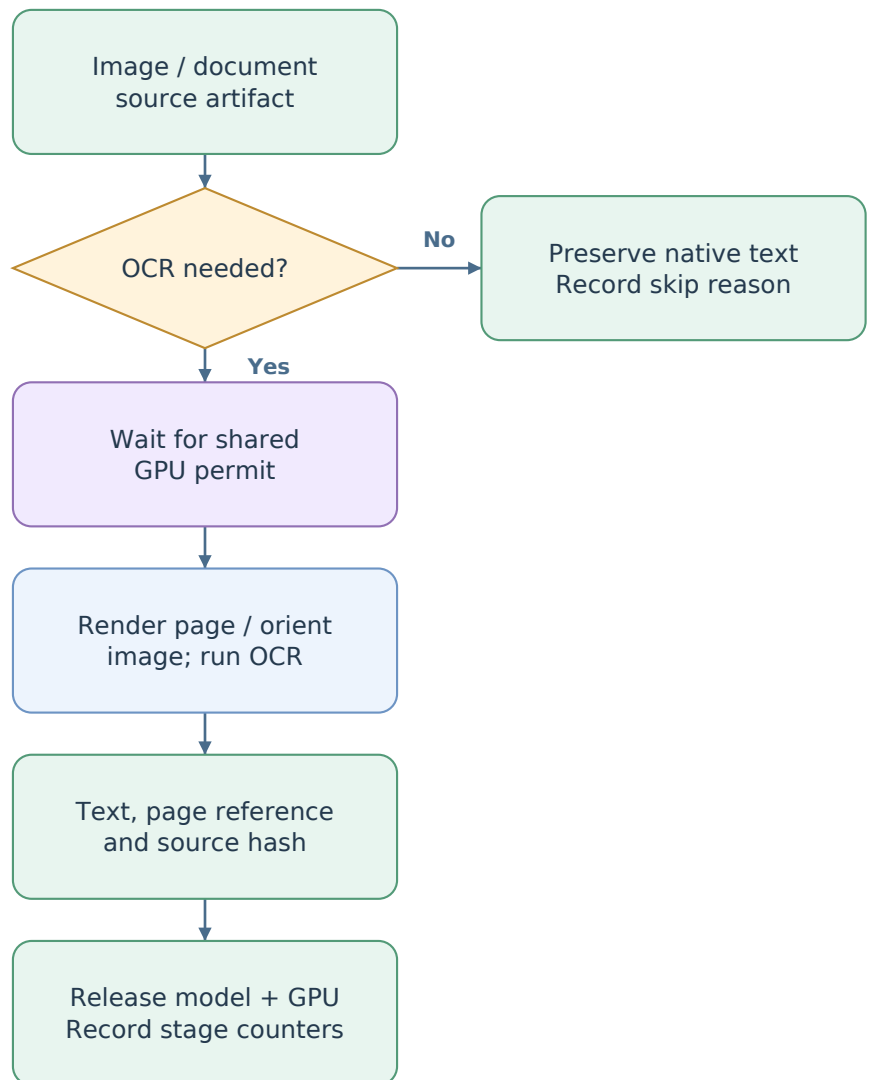
Deleted photos A MediaStore trash row with no acquired image bytes is reference-only. A thumbnail is not the original photo. Retained trash files can be linked when their real bytes and hashes exist.

Deleted messages Residual structures and backup history may yield evidence candidates; they do not guarantee every deleted chat can be recovered. Overwritten, absent or inaccessible encrypted data stays a gap.

Implementation: backend/app/services/mobile_forensic/recovery/analyzers.py;
mobile_forensic/parsers/media_store.py

07 / OCR and searchable text

OCR processes acquired pixels and document pages. It records success, empty results and failures separately.



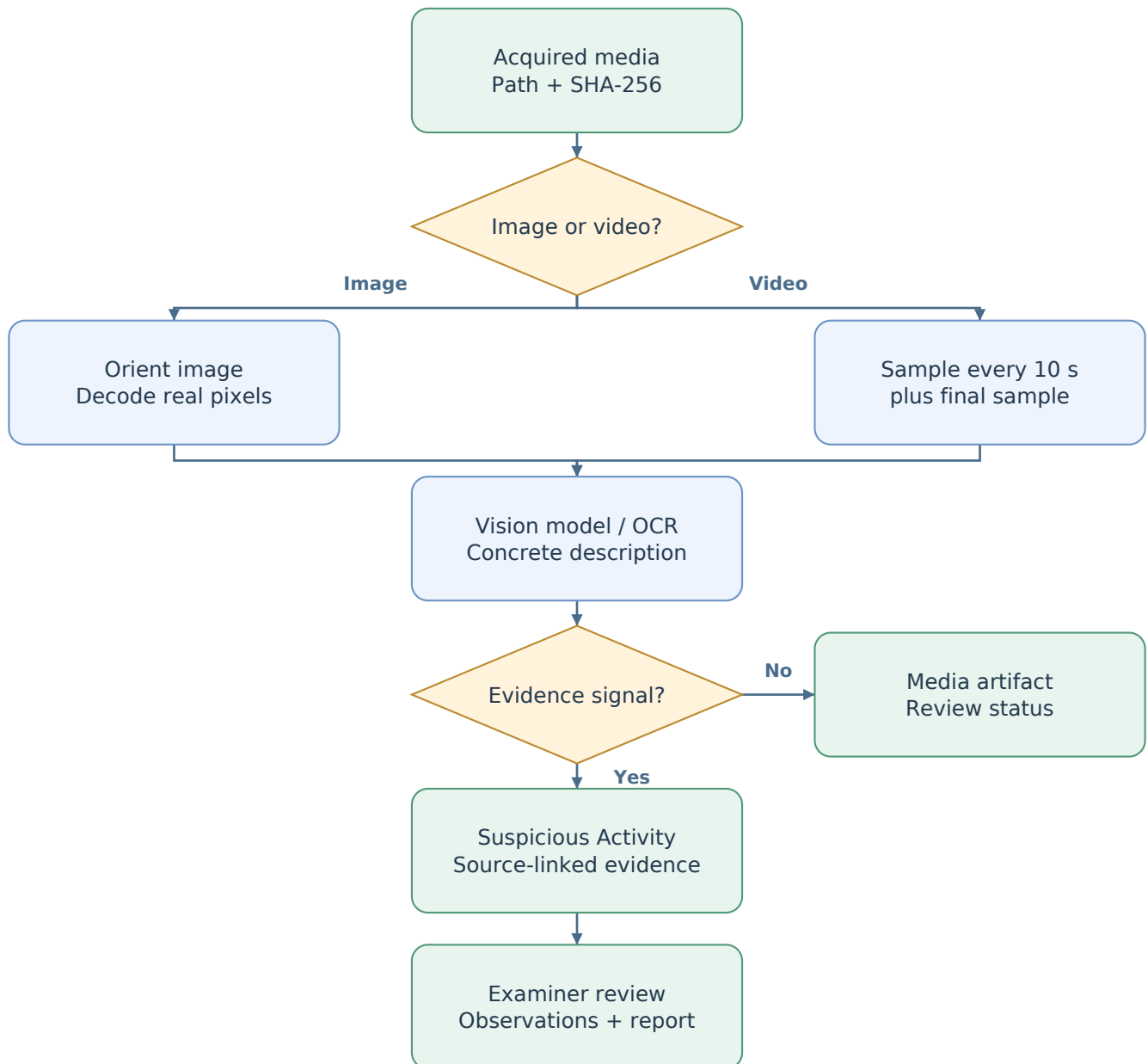
Scheduling OCR and vision share one GPU permit. A legitimate permit or model wait is visible to progressAgent and is not classified as an invalid halt.

Evidence OCR is derived text. Keep the original file, page/frame reference, source hash and OCR status; an OCR guess does not replace the original evidence.

Implementation: backend/app/services/forensic_serial_stages.py; ocr_gpu.py; forensic_media_review.py

08 / Image and video evidence review

Suspicious Activity is created from observable source content and its provenance. Filenames alone do not establish suspicious conduct.



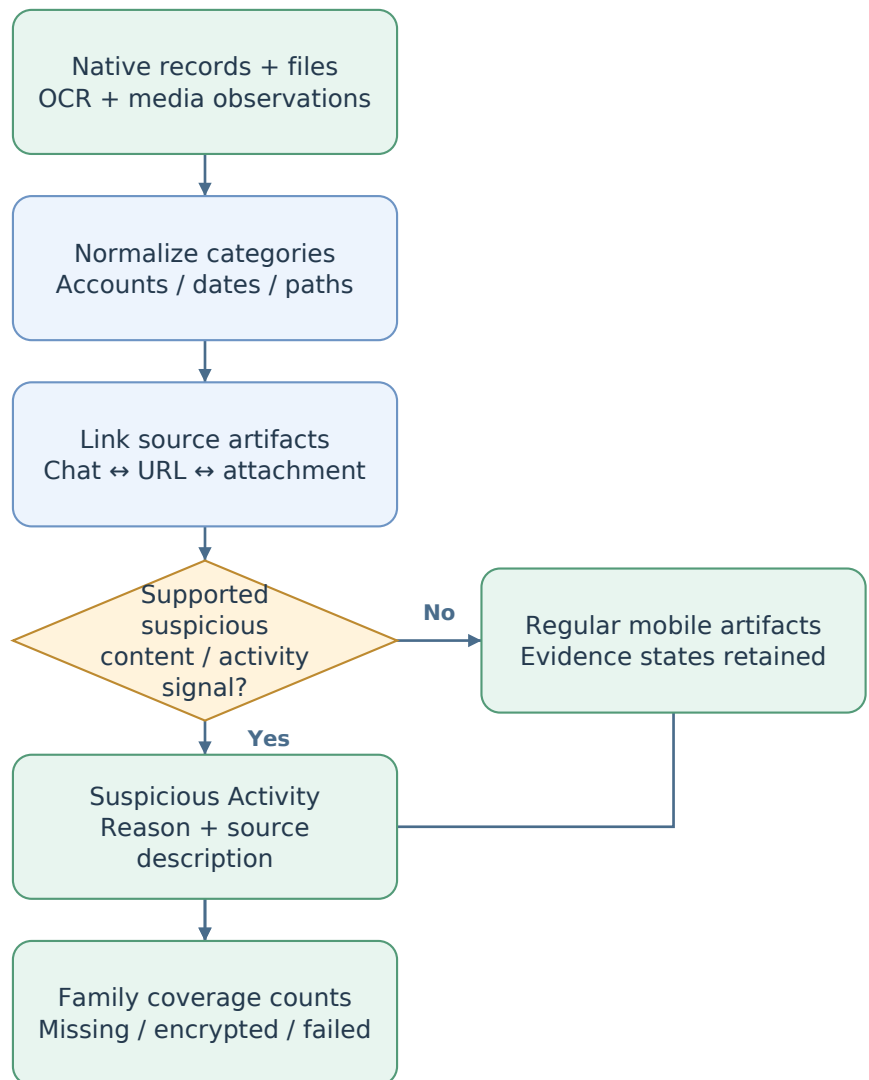
Exact evidence Retain artifact ID, original path/hash, model and description. Video evidence also retains decoded presentation time, derived frame PNG and frame hash.

Scope Images are resized for review up to 1536 px; animated images use the first frame. Video sampling is not an examination of every frame. Missing models or decode failures produce a wait or recorded gap; model interpretation needs examiner review.

Implementation: backend/app/services/forensic_media_review.py; suspicious_activity.py

09 / Mobile inventory and Suspicious Activity

Normalization merges source-backed records into searchable Mobile artifacts and a separate Suspicious Activity category.



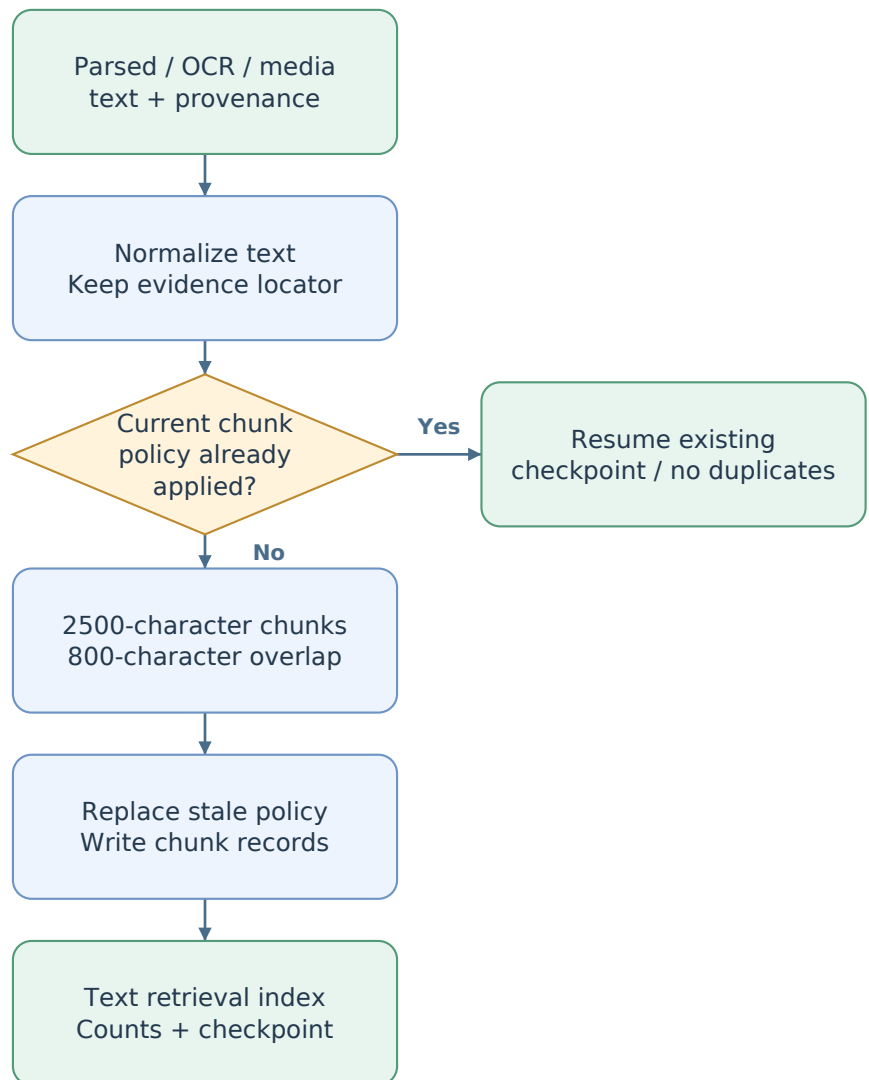
Priority evidence Chats, pictures/videos, downloads, documents, browsing and relevant app records use concrete text, visible media or native events. Source hashes and record locators support every explanation.

Data model `mobile_normalized_artifacts` and `mobile_inventory_items` preserve record-level content and file-level state. Inventory coverage is surfaced instead of claiming success from an APK or encrypted backup alone.

Implementation: `backend/app/services/forensic_serial_mobile.py`; `suspicious_activity.py`; `mobile_artifact_inventory.py`

10 / Text-only RAG chunking

The embedding stage and forensic vector search are removed. The RAG source is normalized text, parsed content, OCR and evidence descriptions.



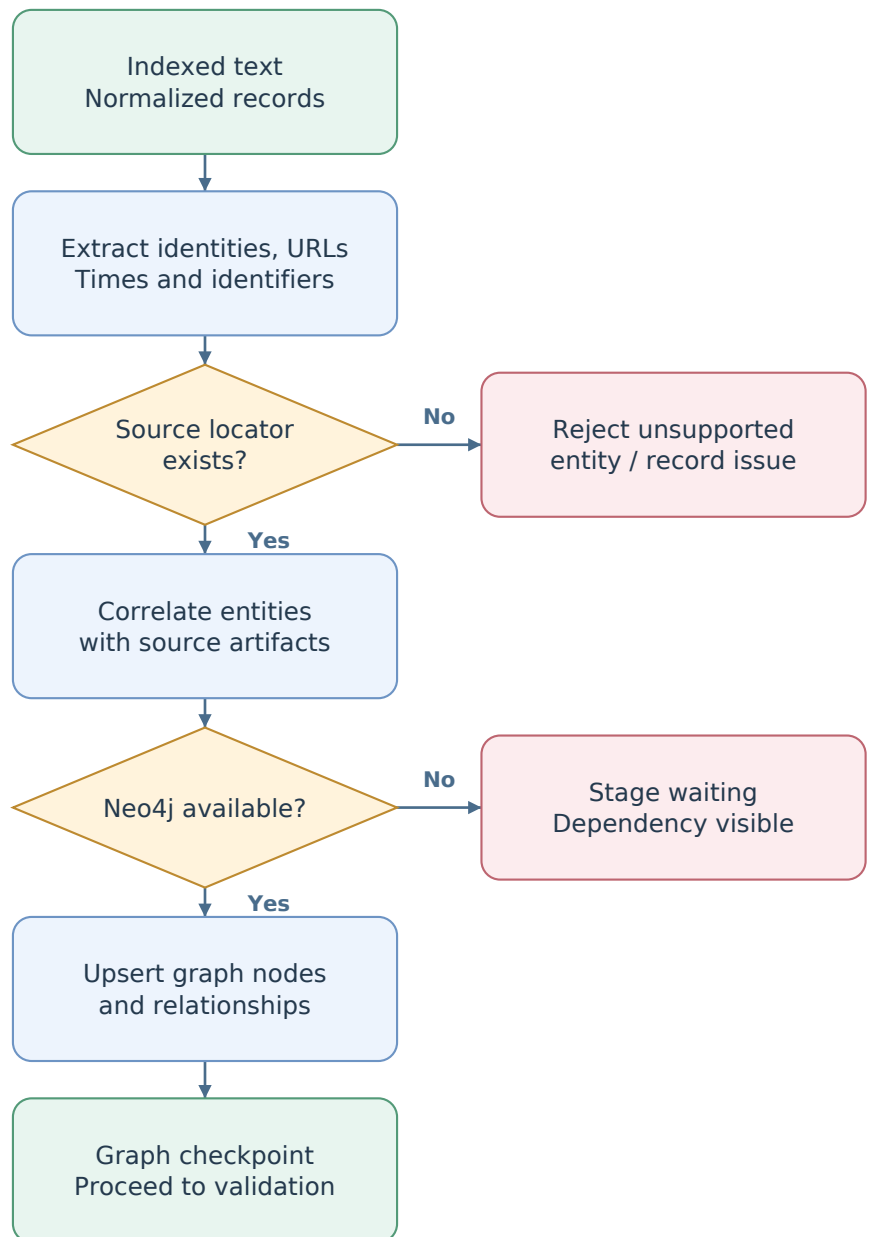
Window definition The limit is characters, not tokens. Headers count inside the 2500-character window. Maximum nominal advance is 1700 characters; boundaries can adjust to the content.

Query / report Retrieval uses text search and evidence references. No GPU embeddings or CLIP retrieval are required. Indexing source descriptions does not add facts beyond the cited source content.

Implementation: backend/app/services/forensic_serial_policy.py; forensic_serial_stages.py; rag_index.py

11-12 / Entities and evidence graph

Enrichment runs after durable chunks. Graph writes preserve source relationships and use idempotent keys.



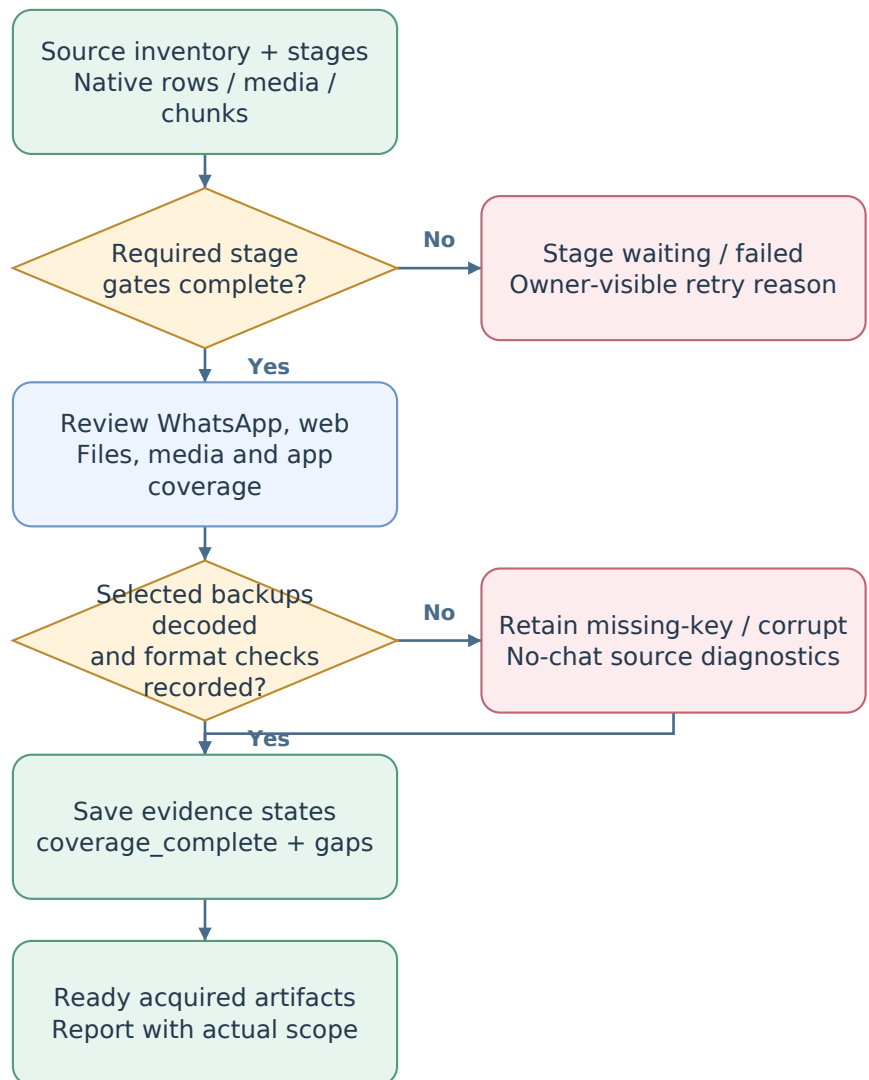
Identity A chat account, sender, image path or URL links to the actual acquired record. Matching names or filenames remain weaker correlation signals.

Dependency gate A graph dependency wait holds the serial gate. `progressAgent` checks the wait reason and retries within policy; it must not report the job ready while the required stage remains waiting.

Implementation: `backend/app/services/forensic_serial_stages.py`; `forensic_serial_pipeline.py`; `forensic_priority_evidence.py`

13 / Mobile validation and coverage

Validation checks successful stages and actual family coverage, including decryption outcomes for the selected backups.



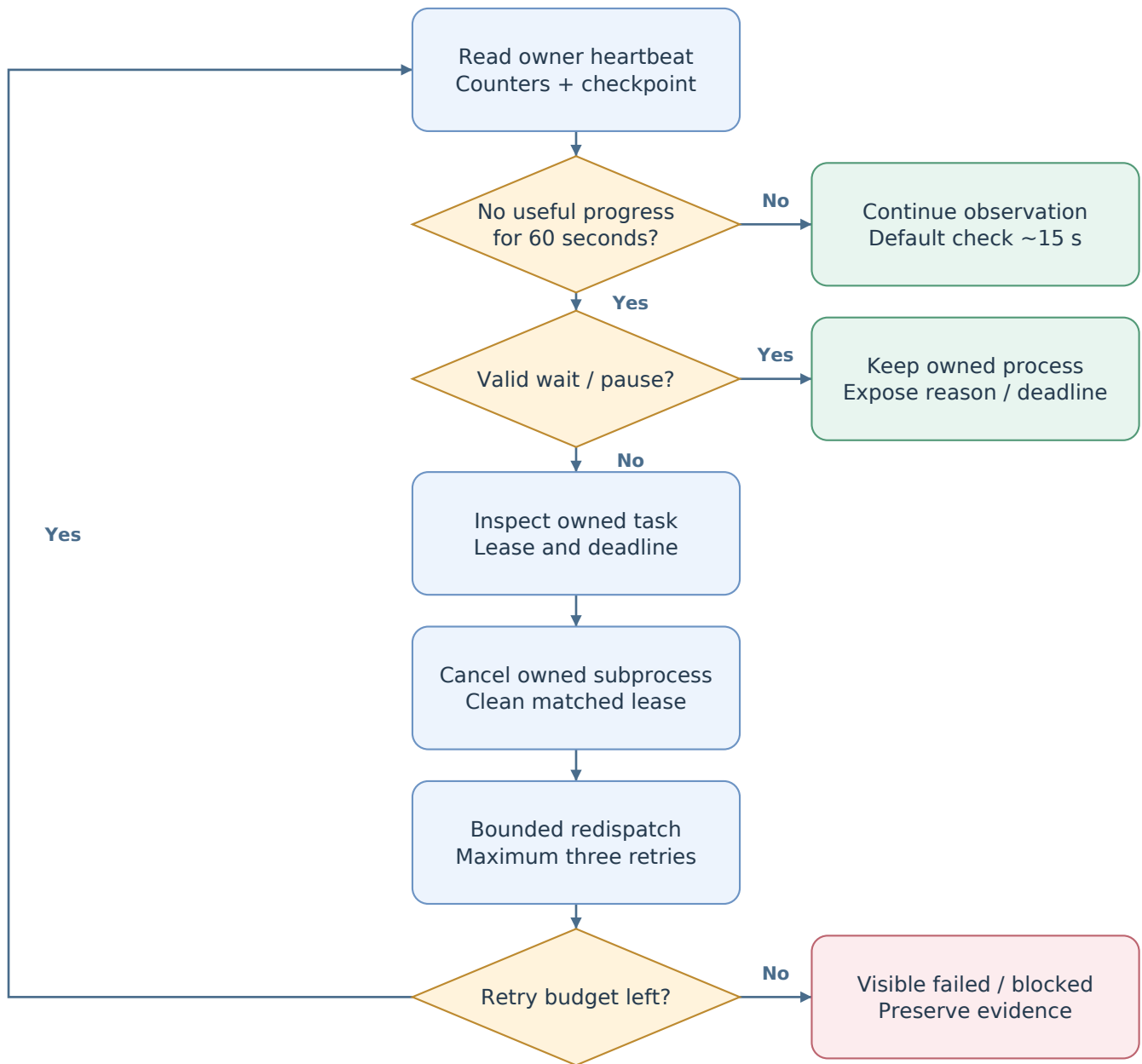
Acceptance meaning Pipeline readiness can coexist with source-access gaps. Modern backups require authenticated recovery. Unauthenticated historical results retain their integrity warning. Live acceptance also requires the intended phone bytes and recoverable records.

Historical imports Reprocess derived stages when a better acquisition or parser becomes available, then generate a new report. Retain the original encrypted backups and historical report runs.

Implementation: backend/app/services/forensic_serial_mobile.py; mobile_forensic/whatsapp_crypt.py; scripts/Test-Live-Phone-GPU.ps1

The single progressAgent supervisor

progressAgent is the sole outside-process coordinator. Observe, Performance and Repair huddles are retired; queued legacy huddles perform no work.



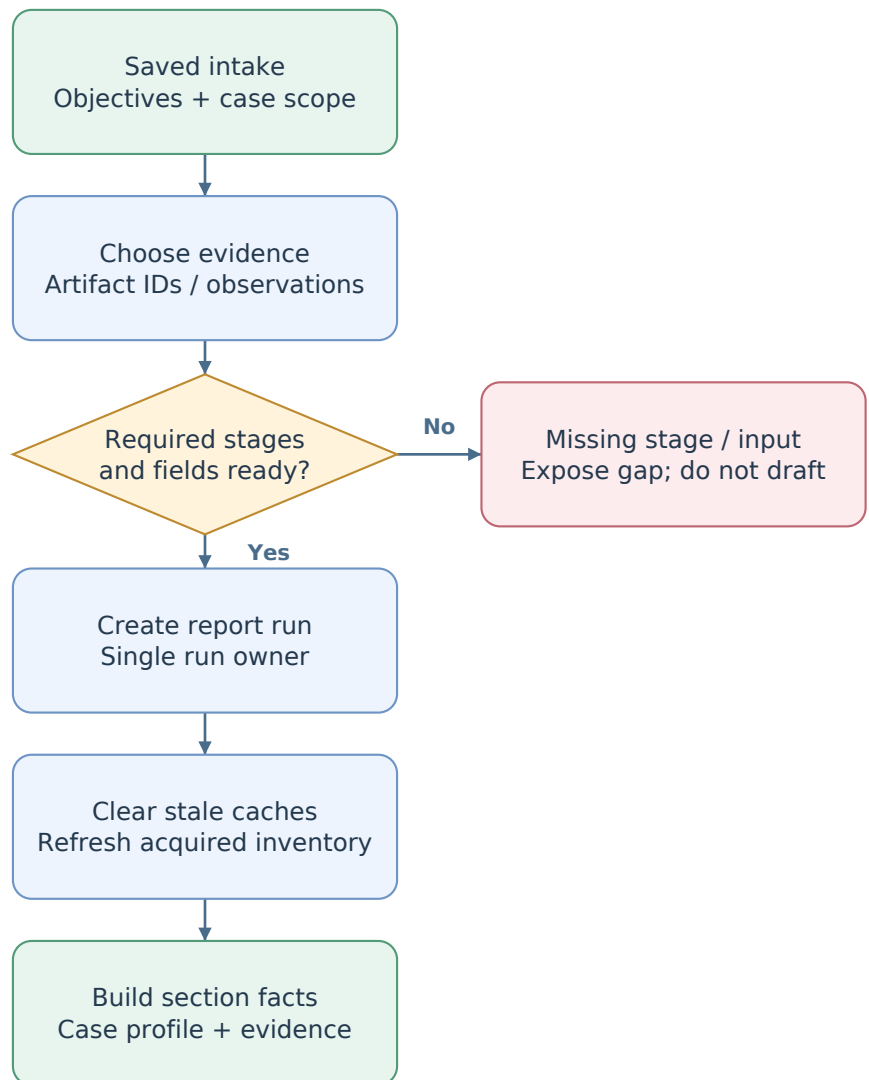
Valid waits User pause, a shared GPU permit, a required model/dependency, or an operation inside its expected deadline may legitimately exceed one minute. A fresh heartbeat alone is not useful progress.

Ownership / cleanup Recovery stops only the owned job process and matched leases. Routine legacy huddle lines are hidden from consoles; historical database rows and processing/evidence logs remain intact.

Implementation: backend/app/services/progress_agent.py; forensic_serial_pipeline.py

Report / Intake, scope and evidence selection

A report run starts from saved case intake and selected evidence. Mobile uses device and extraction details in the shared Disk page structure.



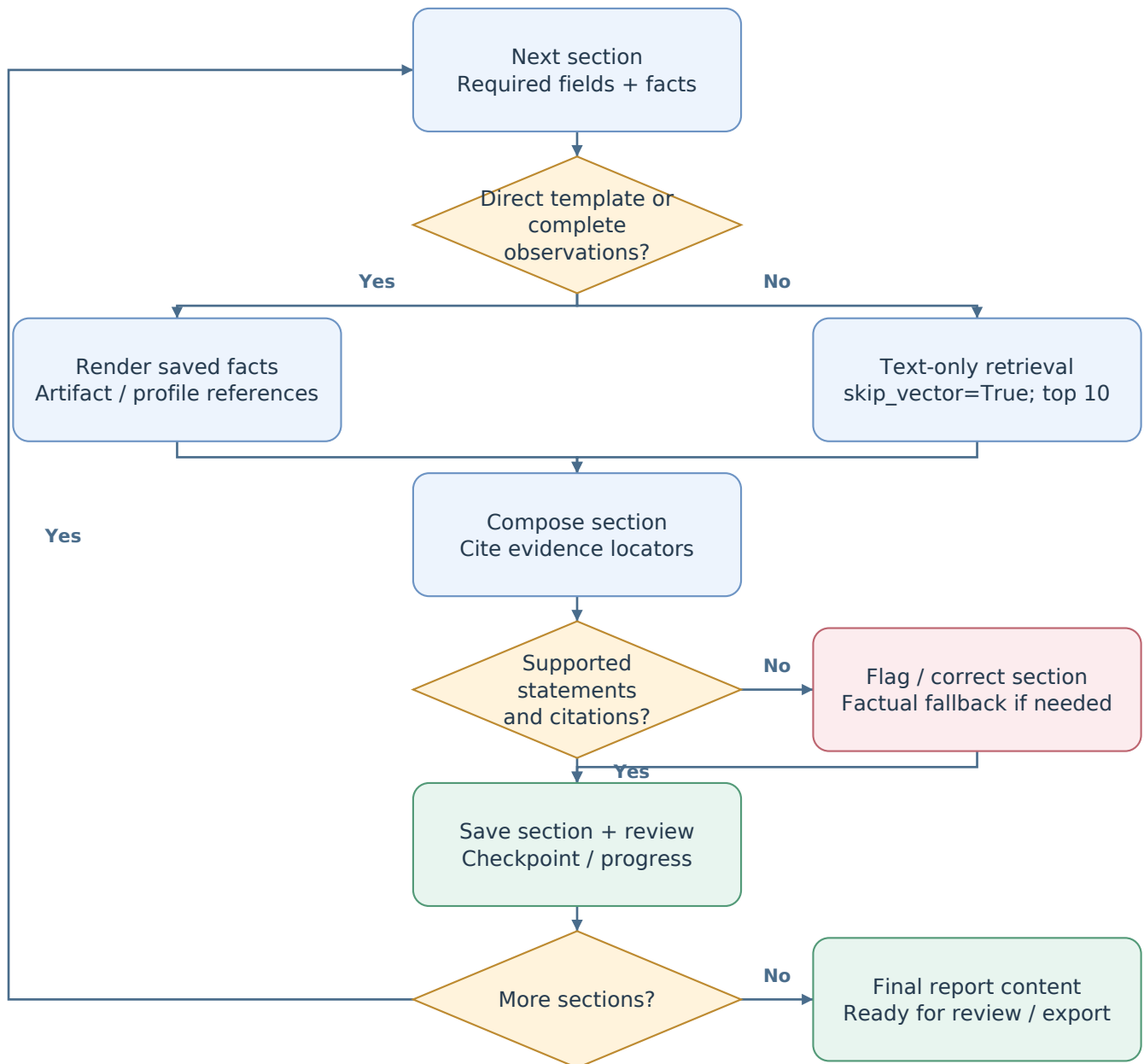
Refresh scope Refresh reads the existing acquisition. Report generation does not acquire a missing phone database or remount an unavailable original source to invent evidence.

Selection Retain selected artifact counts, objectives, procedure and examiner observations. Saved historic reports keep their content; a new run is needed to include new derived evidence.

Implementation: backend/app/routers/report.py; backend/app/services/report_generator.py

Report / Grounded section generation

Sections share a deterministic order. Structured facts and saved observations are preferred where the report has direct evidence.



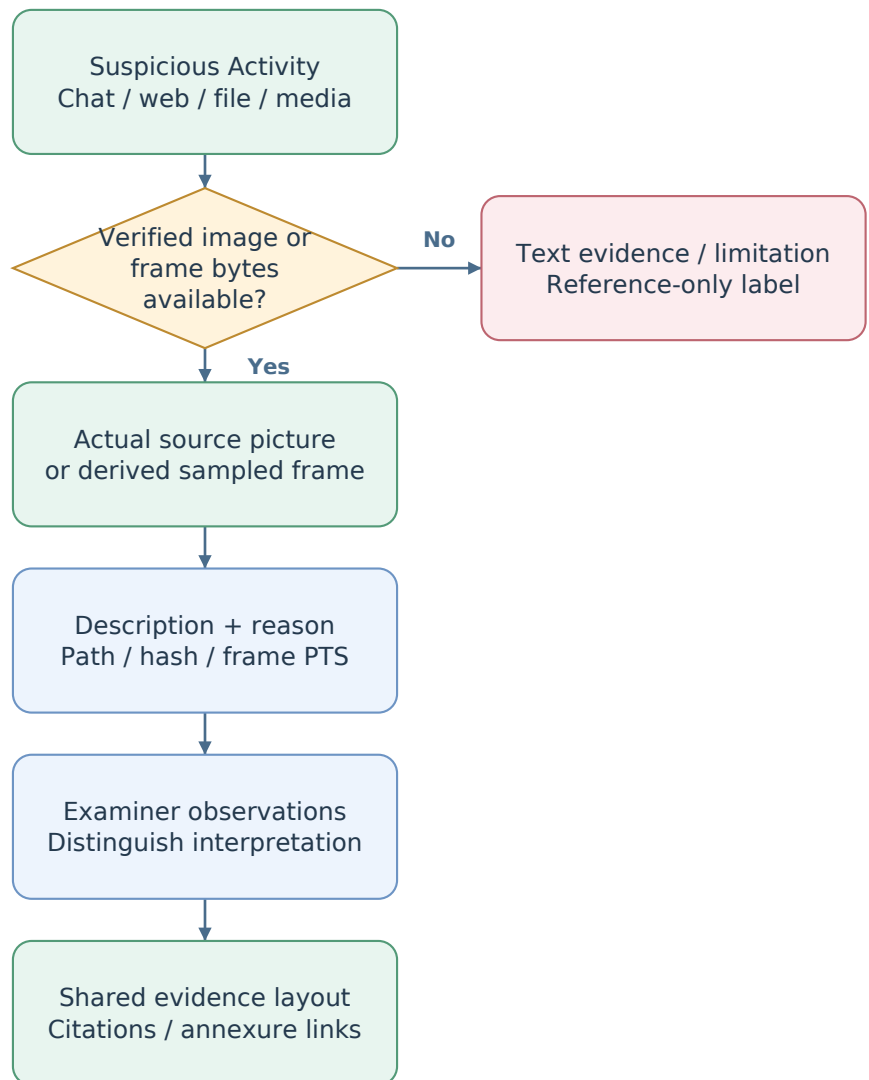
Model boundary The model can describe source facts. Unsupported claims, incomplete evidence and failed generation are reported explicitly; factual templates provide fallback content.

Resuming Section checkpoints and a single report owner support pause/resume. progressAgent tracks actual work, section state and valid model/dependency waits.

Implementation: backend/app/services/report_generator.py; report_generator_agent.py; report_renderer.py

Report / Media evidence and observations

Mobile uses the shared Suspicious Activity section, including verified pictures and source-linked video frames.



Evidence quality Show the actual retained image or derived frame. Identify thumbnails and recovered candidates. Do not substitute illustrative images for missing evidence.

Alignment The section uses the same page width, header, watermark, caption and pagination rules as Disk. Device Information and Extraction Summary replace Disk-specific headings.

Implementation: backend/app/services/report_renderer.py; forensic_priority_evidence.py; suspicious_activity.py

Report / Common section and page structure

The shared renderer keeps the Disk and Mobile report sections aligned. Platform-specific source headings are substituted without changing the surrounding structure.

Report block	Shared content / platform substitution
A / opening	Cover; contents; introduction; scope; tools
A / source	Disk: Forensic Imaging; Mobile: Extraction Summary
A / profile	Disk: OS Information; Mobile: Device Information; User Profile
B	Artifact Summary
C	Objectives, Procedure and Observation
New evidence section	Suspicious Activity
D	Annexure
E	Final Analysis Summary
F	Appendix

Shared formatting A4 page width, section spacing, header, watermark, page numbers, evidence captions and annexure links use the same rendering path.

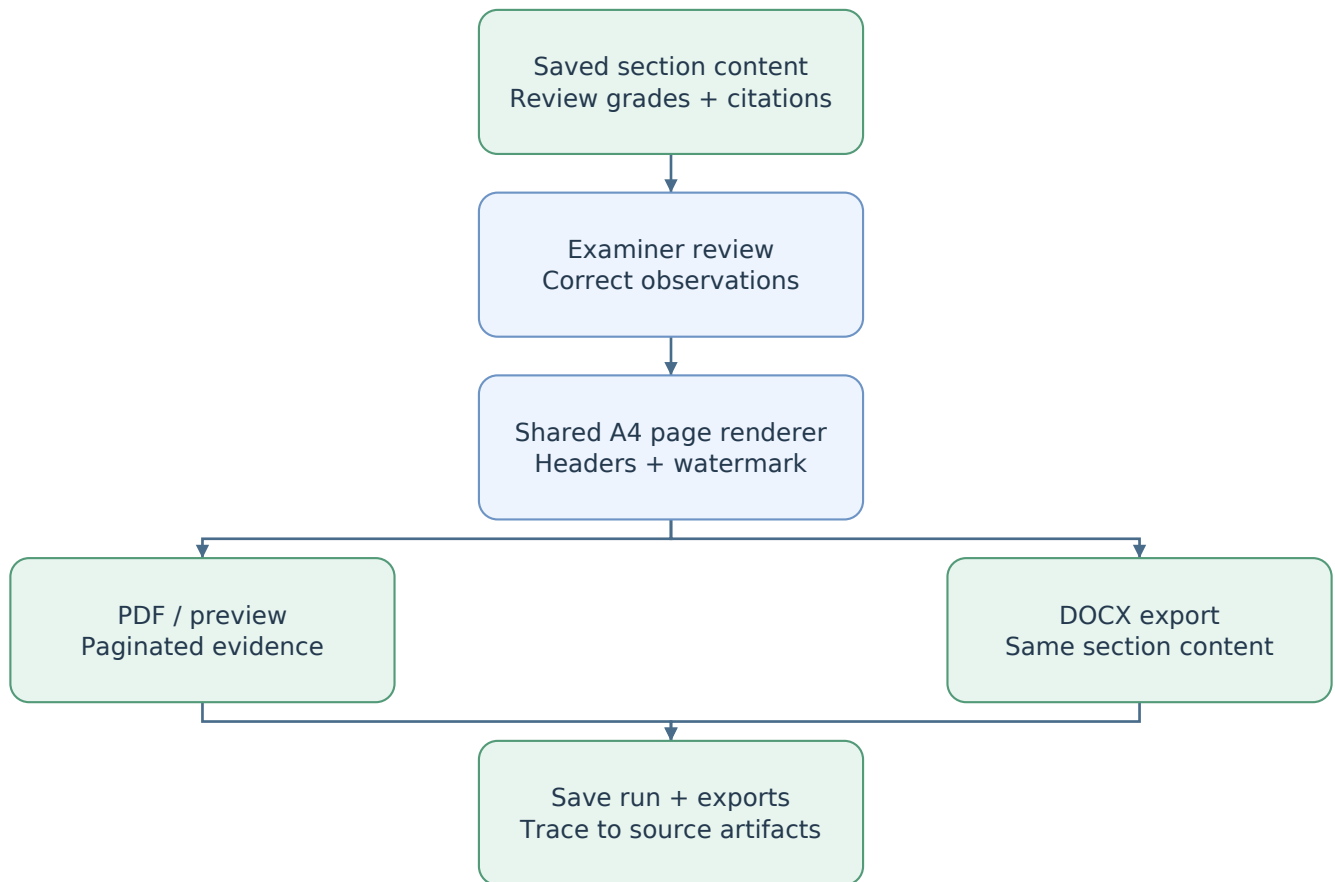
Content and pagination Fourteen logical sections are saved in the report run. Evidence volume determines the page count; identical layout does not imply identical length.

Suspicious Activity This is a dedicated report section and artifact category. It includes source-linked reasons, real retained media or sampled frames, and separate examiner observations.

Implementation: backend/app/services/report_generator.py; report_renderer.py; report_pdf_export.py; report_docx_export.py

Report / Shared layout, review and export

Mobile reports use the shared report renderer and section order. The number of pages changes with the evidence; page styling and alignment are shared.



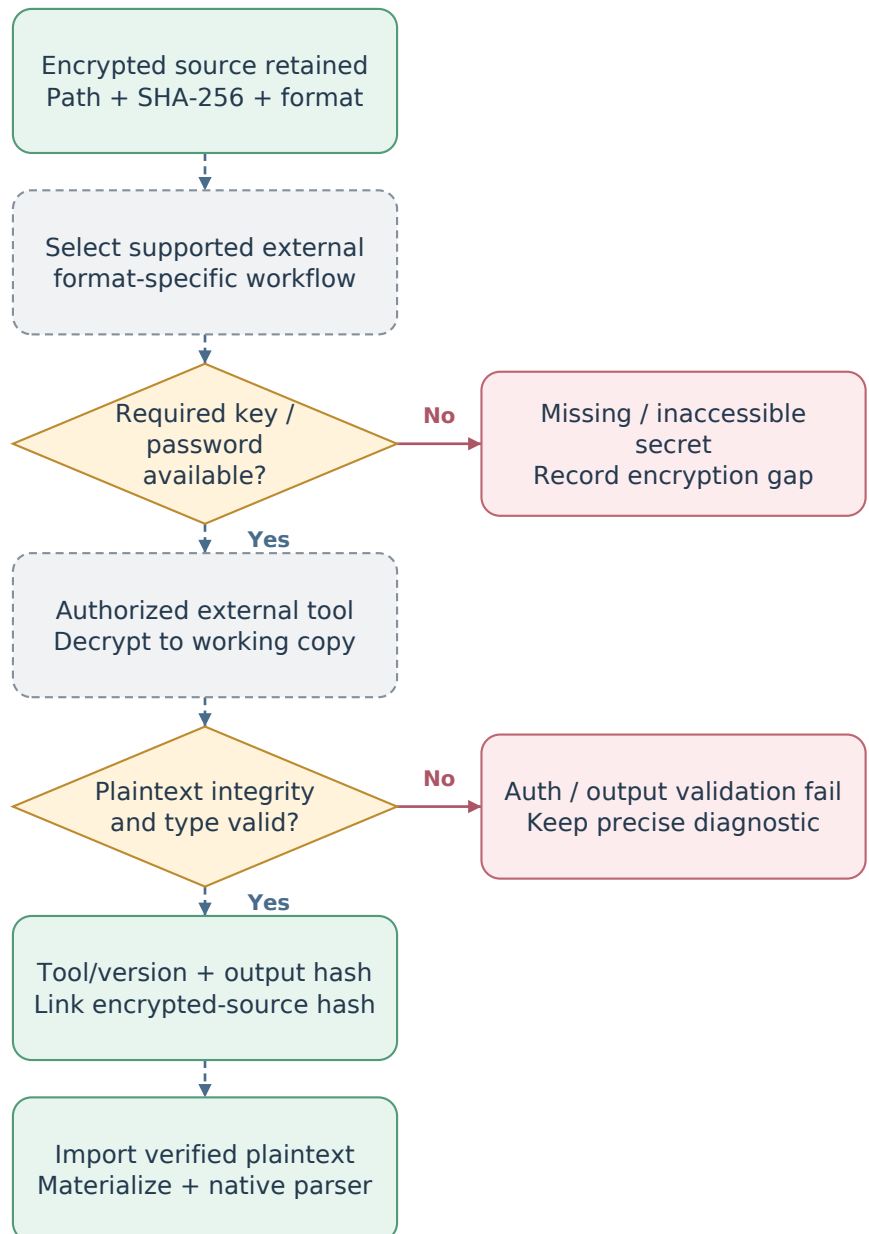
Media alignment Use verified source pictures and derived video frame images with captions, artifact IDs, paths/hashes and frame timestamps. Keep examiner observations distinct from model descriptions.

Limits The shared layout does not force Disk and Mobile reports to have identical page counts. Missing source pixels or unverified deleted-data candidates must not be presented as exact recovered media.

Implementation: `backend/app/services/report_renderer.py`; `report_pdf_export.py`; `report_docx_export.py`

Other encrypted files / External decryption and import

The current package inventories these encrypted sources. A format-specific authorized external tool must produce verified plaintext before the native parser can analyze it.



Applicable routes Signal SQLCipher uses its database key and a compatible SQLCipher workflow; encrypted iOS backups use the correct backup/keybag route; archives, Office/PDF and encrypted mail require their specific supported key/password tools.

Device encryption For Android FBE, acquire accessible plaintext through the authorized unlocked OS or a supported provider. A WhatsApp backup key does not unlock a raw phone filesystem. No acquisition or external decryption was performed for this diagram.

Implementation: `mobile_forensic/parsers/messaging.py`; `mobile_adapters/import_ios_backup.py`; `mobile_forensic/whatsapp_crypt.py` (WhatsApp-only implemented path)

Encryption support and acquisition matrix

These are current package capabilities, not a promise that every encrypted format can be decrypted.

Source / container	Current path	Required condition / failure result
Plaintext chat DB + companions	Native parser / derived replay	Valid SQLite bytes; preserve DB/WAL/journal hashes
WhatsApp crypt7/8/9/10/11/12/14	CBC / authenticated or historical GCM	Matching files/key; legacy structural checks or modern authenticated payload
WhatsApp crypt15	Implemented HKDF + AES-GCM	Matching backup base secret; supported layout and tag
Missing / wrong WhatsApp key	Encrypted backup artifact	No plaintext chats; record key / auth diagnostic
Signal SQLCipher	External authorized decryption	Matching SQLCipher key and supported tool; verified plaintext import
Encrypted iOS backup	External supported decryption	Correct backup password / keybag route; importer itself is not full decryption
Android filesystem encryption	Authorized unlocked OS acquisition	Accessible private source or decrypted full dump; WhatsApp key is insufficient
Encrypted mail / documents / archives	External supported decryption	Correct format-specific key/password; retain encrypted-source provenance
WhatsApp web / other encrypted apps	Inventory / supported external route	Actual database/session keys and verified plaintext; no E2EE bypass

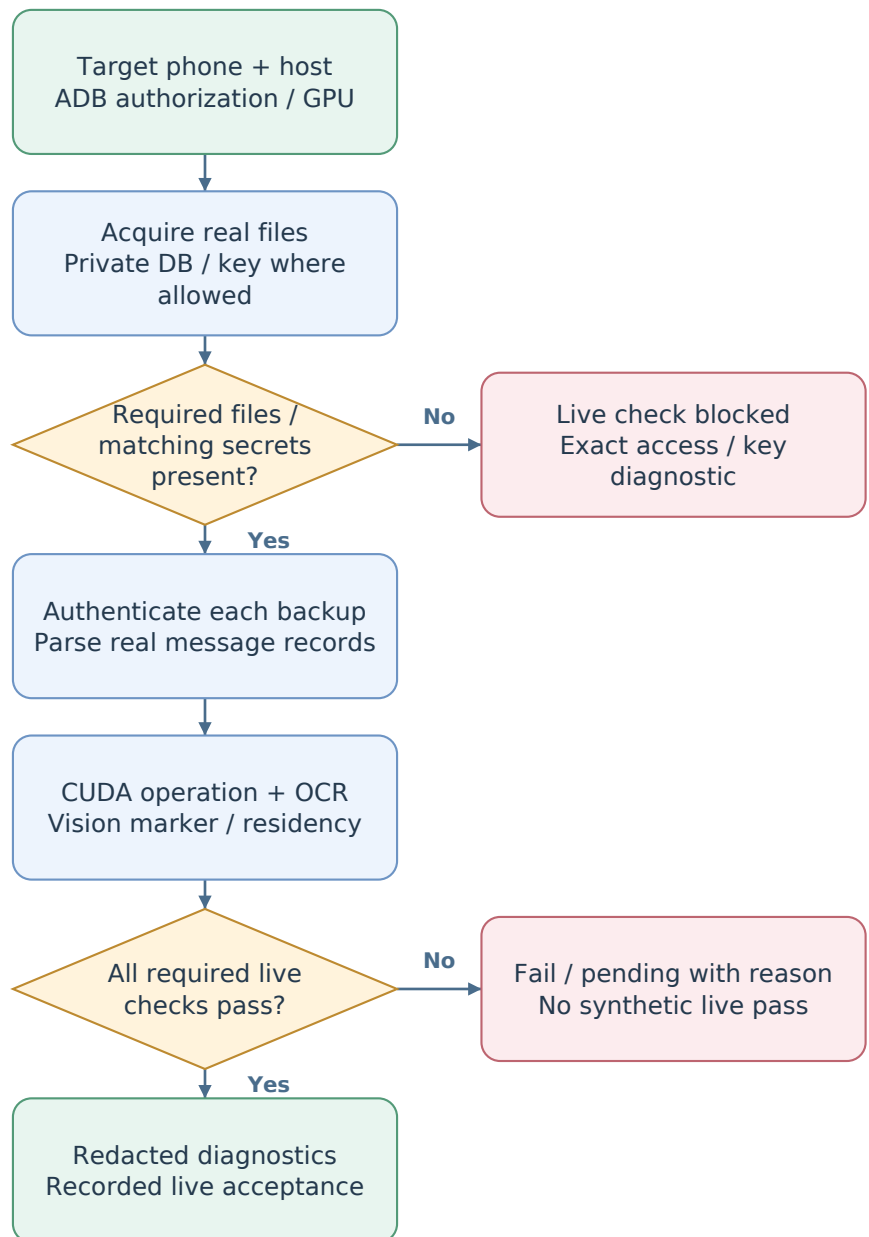
Re-import boundary Original .crypt uses its historical format key. CRYPT5 accepts an already-derived 24-byte key or its original account-email route; see the final format/test matrix. For external tools, document encrypted hashes, tool/version and verified output provenance before import.

Secrets Do not put raw keys, passwords or chat content into acceptance logs or diagnostic bundles intended for sharing.

Implementation: mobile_forensic/whatsapp_crypt.py; mobile_forensic/parsers/messaging.py; mobile_adapters/import_ios_backup.py

Live phone / GPU acceptance and performance

Source and regression checks do not replace a real Android phone, matching encrypted backups or live GPU inference.



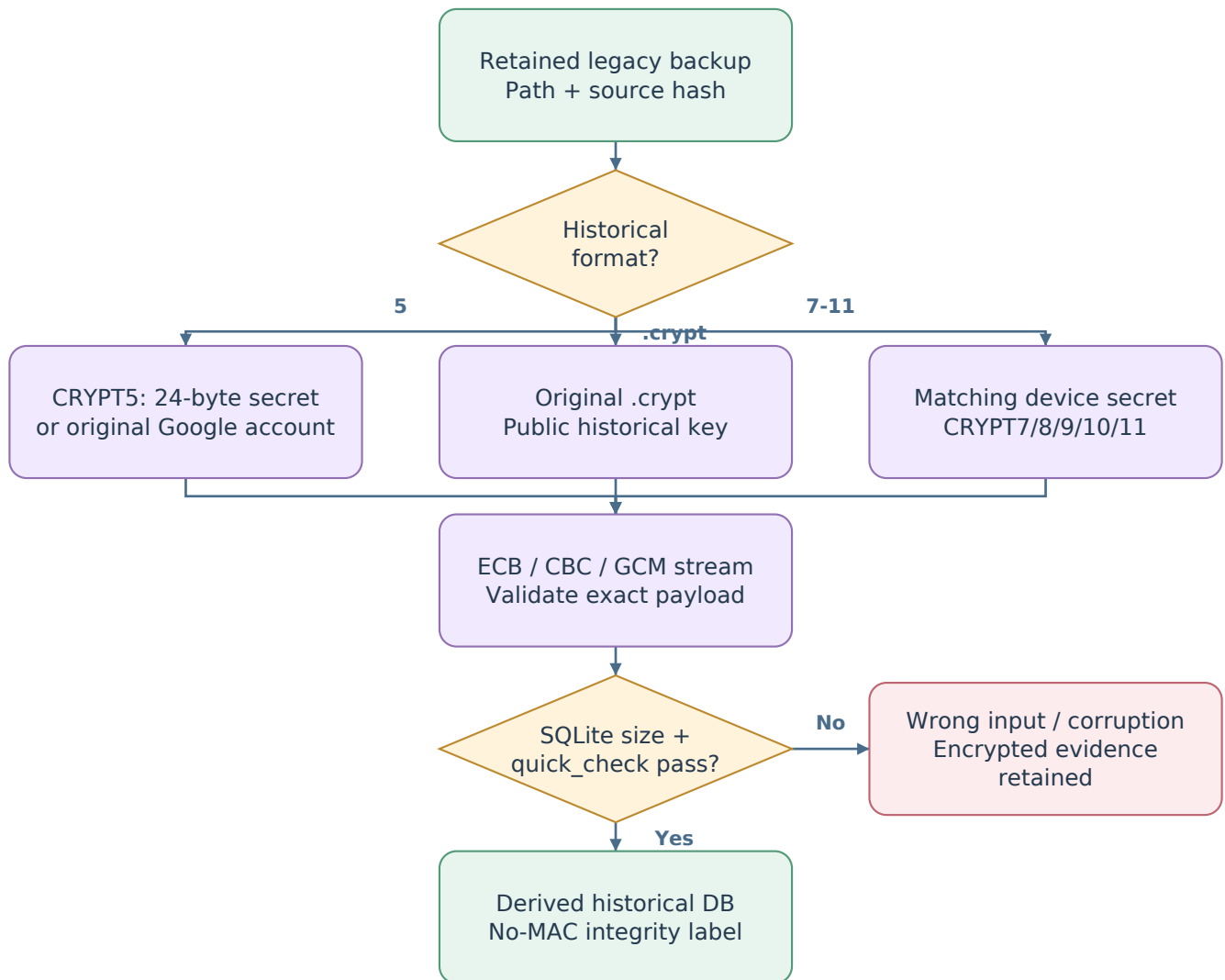
Current status No phone/GPU hardware session was available for this documentation and scanner review. Live acceptance remains pending; encrypted WhatsApp backups still need matching keys.

Performance in code Priority key acquisition, documented protobuf IVs, source/key-set caching, shared-key handling, deduplicated derived databases, bounded ZIP expansion, batched records and 2500/800 text chunks bound resource use.

Implementation: scripts/Test-Live-Phone-GPU.ps1; LIVE-PHONE-GPU-VERIFICATION.md; forensic_serial_mobile.py

WhatsApp / Historical CRYPT formats

Historical ECB, CBC and tagless stream backups do not establish cryptographic authenticity. Recovery verifies SQLite structure and records the integrity limit on every parsed chat.



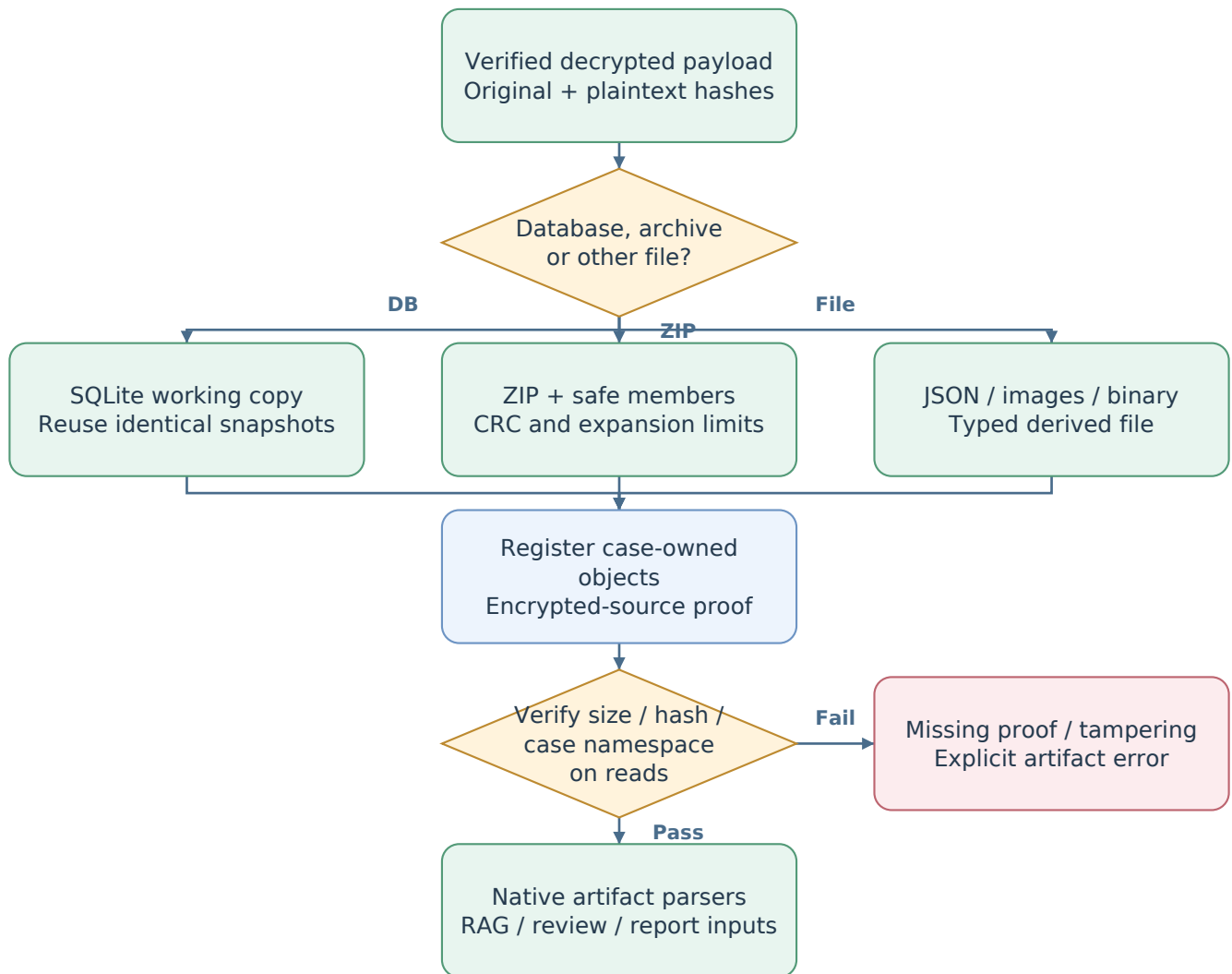
Meaning Structural checks do not prove authenticity. Legacy chat records carry a medium confidence ceiling and the legacy_no_authentication_tag validation note.

Account CRYPT5 account derivation is specific to this old format. It does not extract a CRYPT14 key from ciphertext.

Implementation: mobile_forensic/whatsapp_crypt.py; integrity.py; parsers/messaging.py; frontend JobIntakePage.tsx

WhatsApp / Derived payloads before artifact parsing

The materialize stage decrypts registered case bytes before native parsing, OCR, media review and text-only RAG. Original encrypted files remain in the case.



ZIPs Retain the authenticated archive even if a member cannot be expanded. Reject traversal, Windows device names, symlinks, duplicate names and separately encrypted members. Report limits explicitly.

Source boundary After extraction finalization, the server reads sealed sources or verified registered derivations. It never reopens the original phone/dump folder for a newly created working copy.

Implementation: whatsapp_derivation.py; forensic_serial_stages.py; sqlite_counts.py; artifact_parse.py; artifact_preview.py

WhatsApp / Supported formats and executable test case

The public QA files are synthetic. The CRYPT14 case includes its matching 158-byte key file; the keyless case must remain blocked.

Format	Required material	Validation / result
Original .crypt	Historical public AES format key	AES-192-ECB; complete SQLite structural checks; authenticity unverified
CRYPT5	Already-derived 24-byte / 48-hex key, or exact original Google account	AES-192-CBC; complete SQLite + quick_check; no authentication tag
CRYPT7 / CRYPT8	Matching files/key or 32-byte AES secret	AES-256-CBC; CRYPT8 gzip; complete SQLite checks; authenticity unverified
CRYPT9 / 10 / 11	Matching files/key or raw 32-byte AES secret	GCM tag when present; exact-EOF historical tagless streams remain unverified; failed tags never discarded
CRYPT12 / CRYPT14	Matching files/key; raw / hexadecimal secret accepted	AES-GCM tag and any MD5 footer required; bounded expansion
CRYPT15	Matching encrypted_backup.key or 64-character root secret	HKDF-SHA256 then AES-GCM authentication and footer validation
Other CRYPT extensions	No validated decoder in this release	Retain encrypted bytes; unsupported_crypt_version; never assume another format

Run From the updated project: `python scripts/test-whatsapp-decryption.py --out C:/Aetheris-QA/whatsapp-case` (choose an empty output folder).

Checks The script checks ten formats, typed payloads, key capture offset, keyless blocking, chats, a surviving deleted row, attachment reference, RAG-ready text and unchanged input hashes.

Live status Synthetic and PostgreSQL-compatible checks passed. No connected Android phone, native Windows device acquisition or GPU session was available for live acceptance.

Implementation: `scripts/whatsapp_qa_fixtures.py`; `scripts/test-whatsapp-decryption.py`; `backend/tests/test_whatsapp_payloads.py`